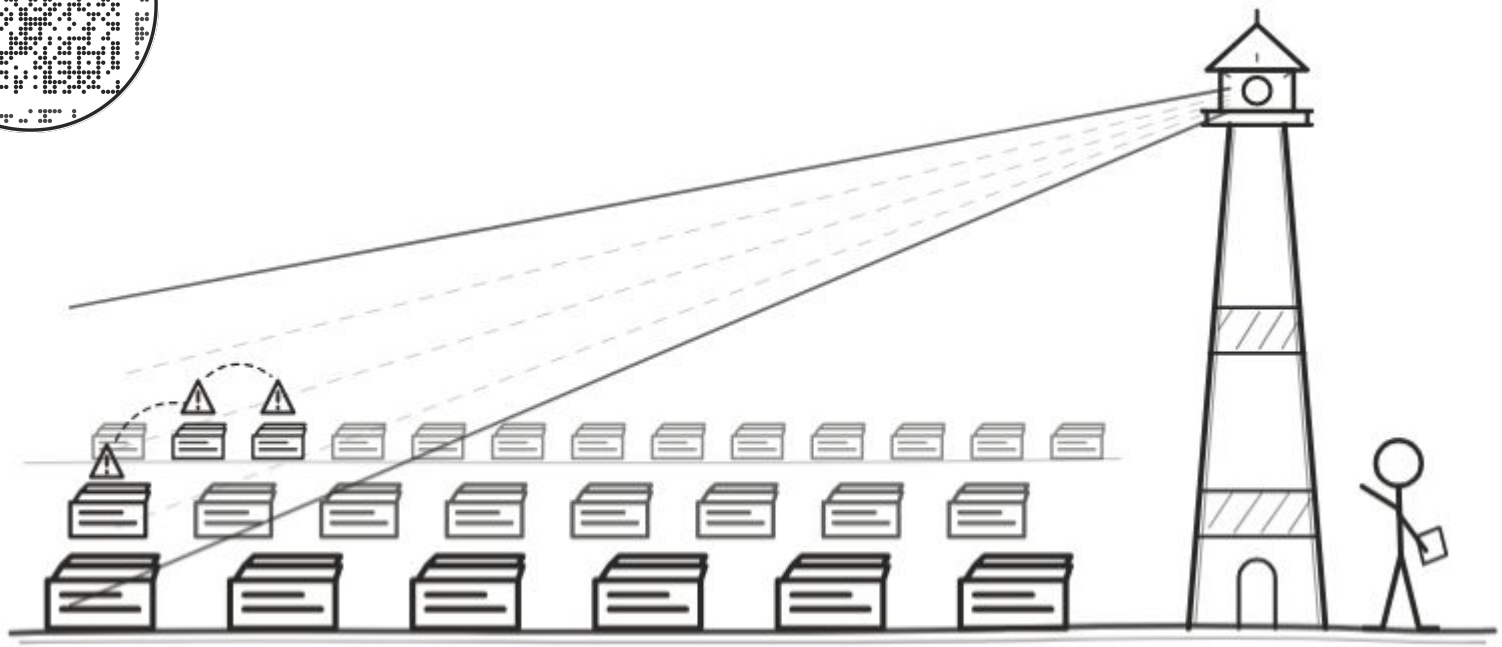
Managing millions of sboms

vulnerability correlation at scale with **trustify**



Oct 2026 Praha - https://openssl-conference.org/

About Me



Work at Red Hat

jfuller@redhat.com | jim@fuller.link

Work (ed | ing) on geophysics, network, databases, spec, security *stuff*

Here to talk about challenges of managing vulnerability correlation at scale using trustify.

***Caveat**: some of my opinions are biased by current experience with Red Hat data*

About You





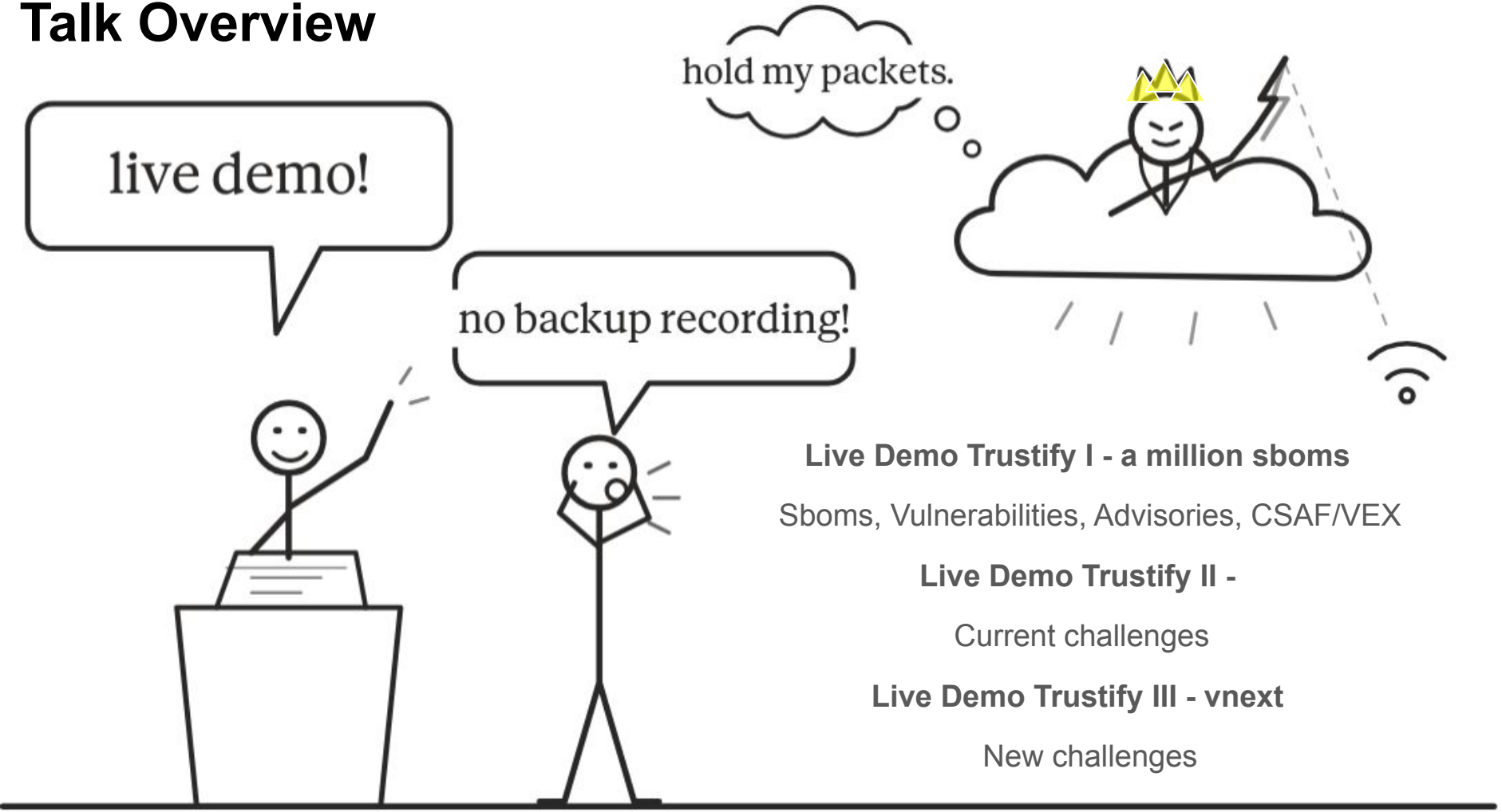

- Develop software and need to identify vulns during development and publish SBOMs & advisories
- Familiar with sboms (cdx,spdx), CPE's, pURL, CSAF, VEX, OSV …at scale everything is hard
- Have sboms incoming from all directions and trying to make sense of it all
- Trying to mitigate false negative|positive vulnerability correlation from the data you have
- Have discovered that an SBOM can be perfectly schema valid and still tell you nothing useful…
- Have discovered that pURLs are not world wide unique opaque identifiers …
- Might have found a statically linked copy of something in a binary that no SBOM anywhere records
- Might know that version comparison lies to you the moment backported fixes enter the picture
- Might be aware that CRA has become someone's problem recently … might even know Directive (EU) 2024/2853 is getting enforced end of year …
- Might know confidence and provenance is the answer to everything but no one has it …
- Might understand that non-deterministic processing works better when paired with deterministic 'oracles'
- Other talks were full …

You know why you are here … I am curious about your own experiences/questions/advice ;)






Talk Overview



Live Demo Trustify I - a million sboms

Sboms, Vulnerabilities, Advisories, CSAF/VEX

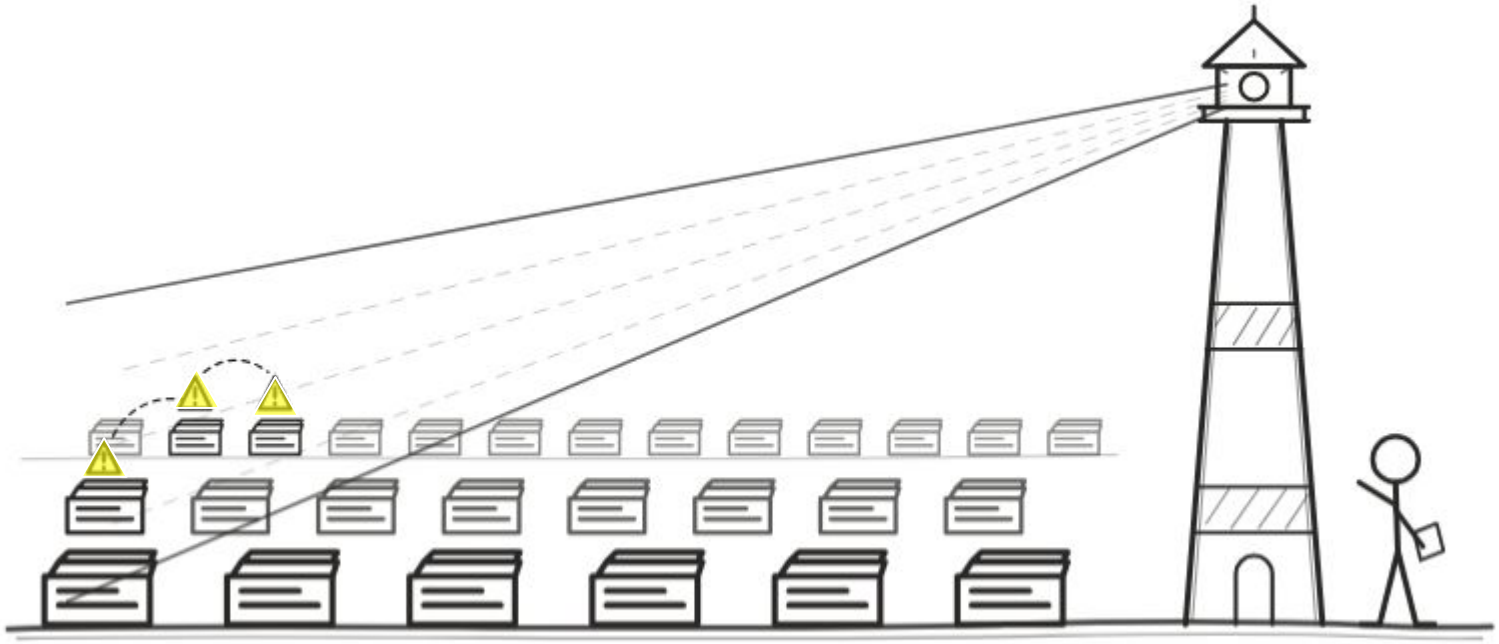
Live Demo Trustify II -

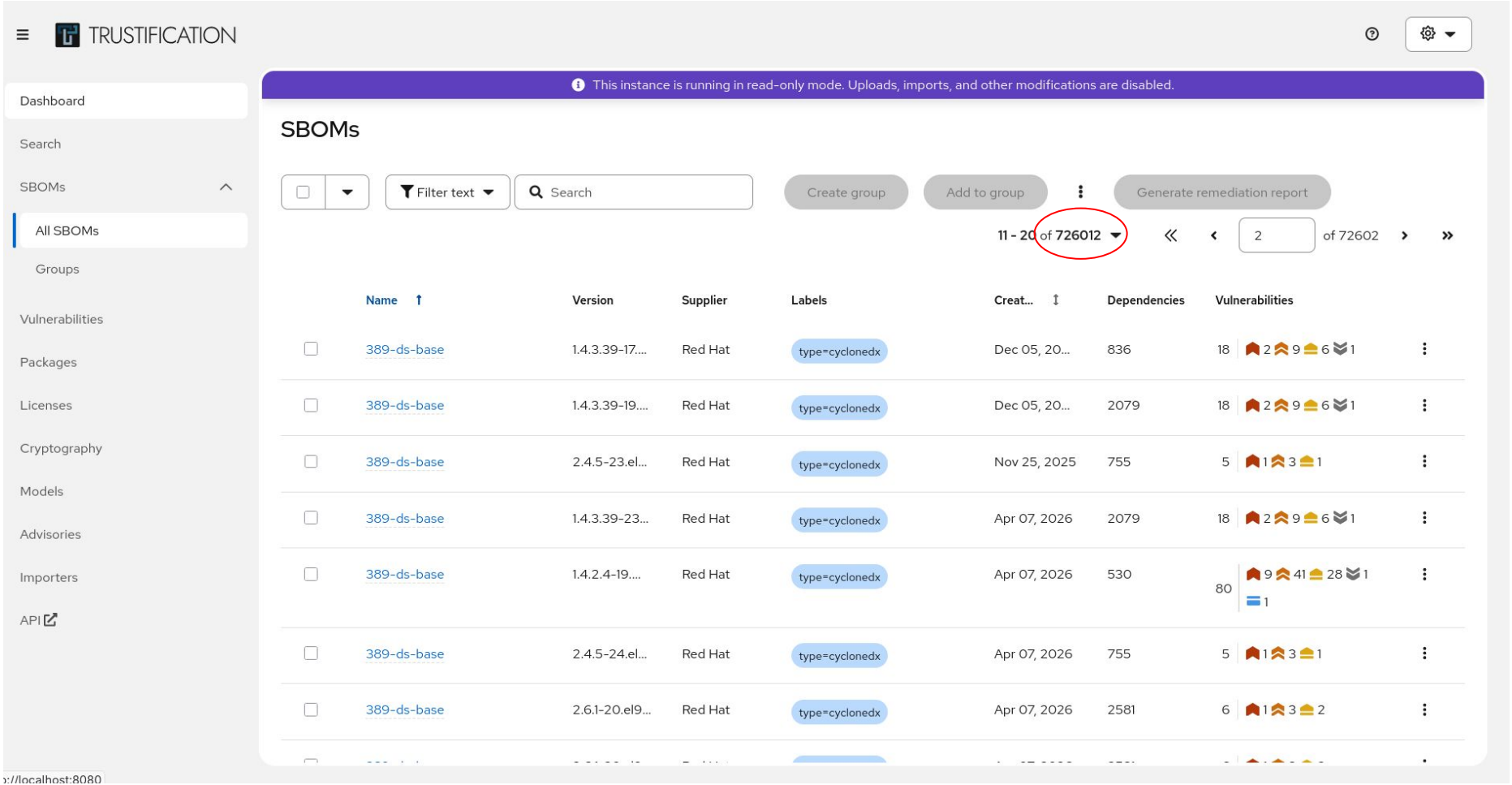
Current challenges

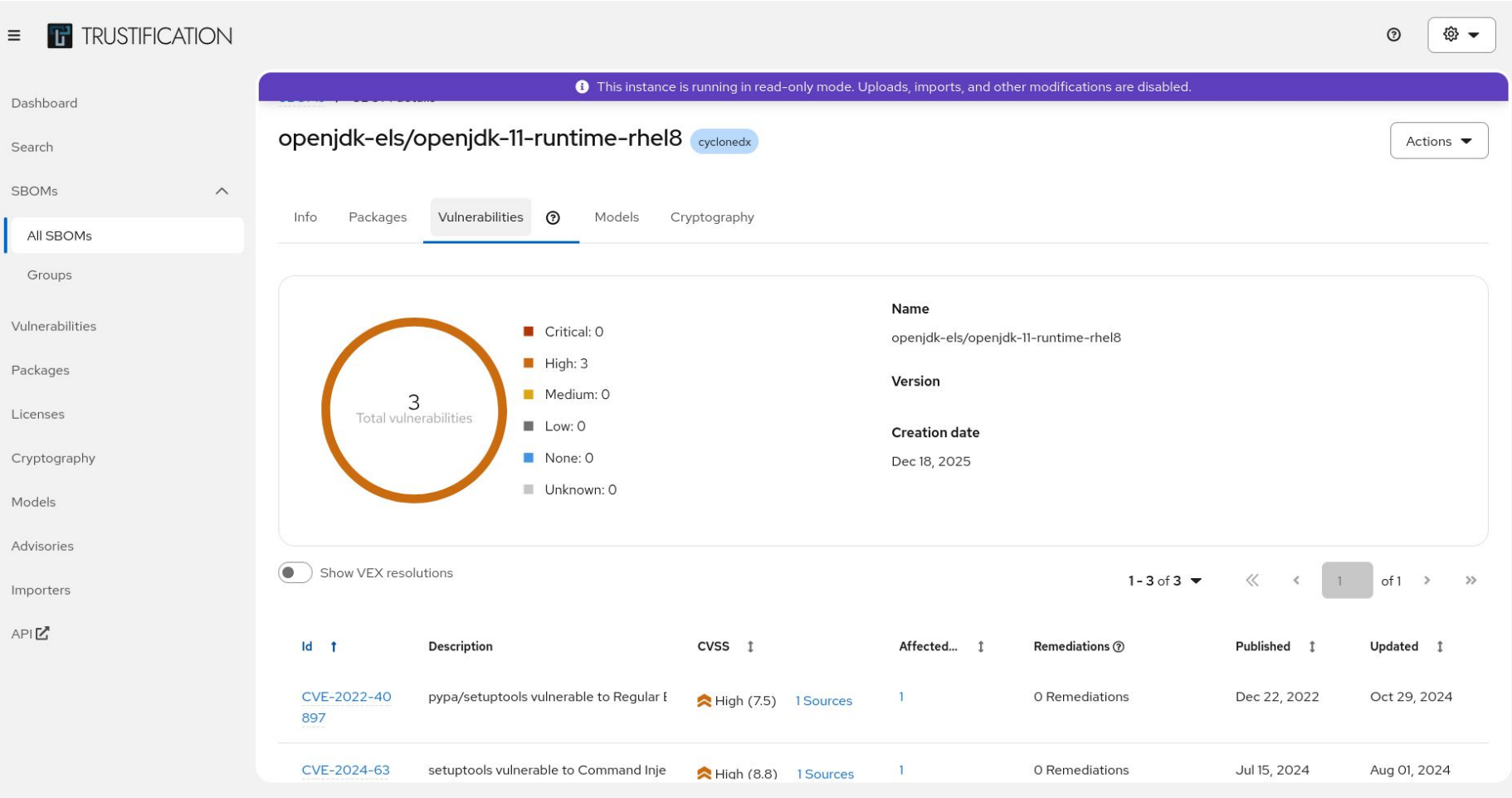
Live Demo Trustify III - vnext

New challenges

Demo trustify I



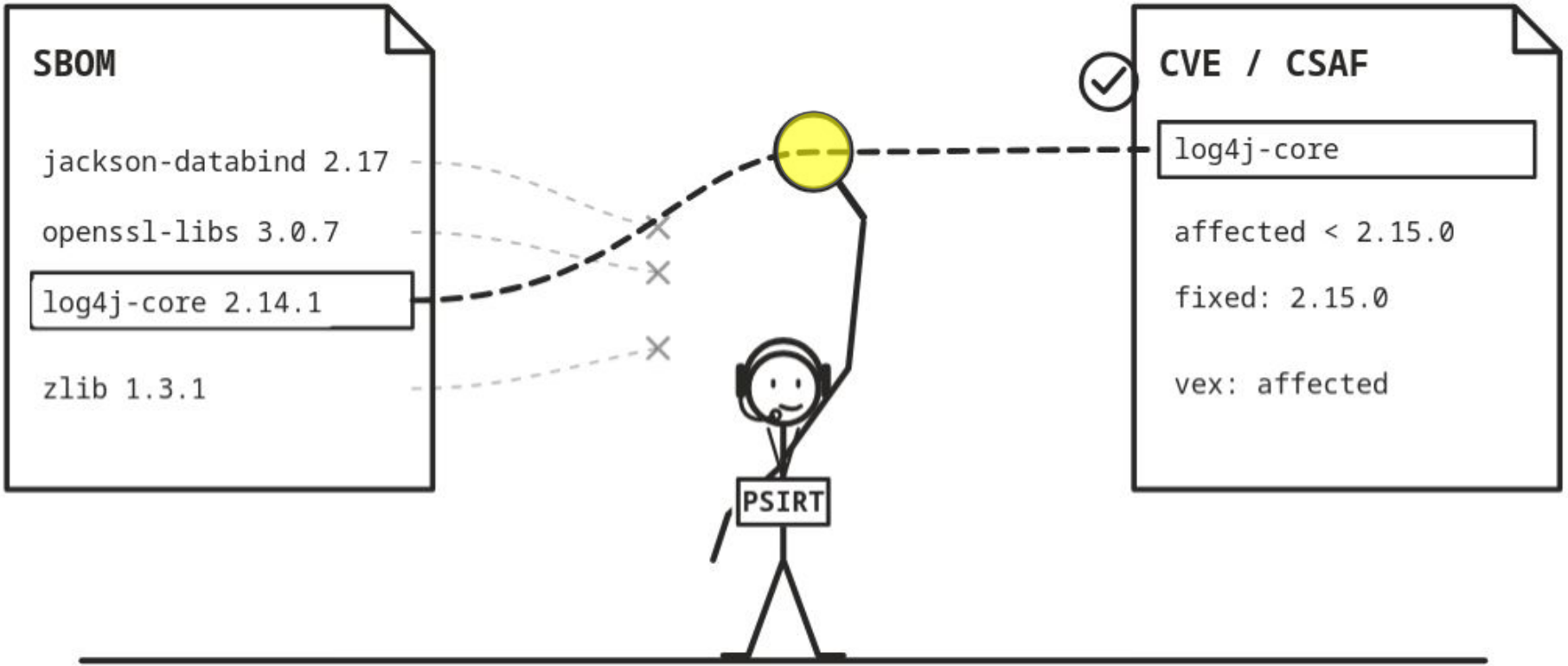

TRUSTIFICATION
This instance is running in read-only mode. Uploads, imports, and other modifications are disabled.
Dashboard
Search
SBOMs
All SBOMs
Groups
Vulnerabilities
Packages
Licenses
Cryptography
Models
Advisories
Importers
API
SBOMs
Filter text
Search
Create group
Add to group
Generate remediation report
11 - 20 of 726012
2
of 72602
Name
Version
Supplier
Labels
Creat...
Dependencies
Vulnerabilities
389-ds-base 1.4.3.39-17.... Red Hat type=cyclonedx Dec 05, 20... 836 18 2 9 6 1
389-ds-base 1.4.3.39-19.... Red Hat type=cyclonedx Dec 05, 20... 2079 18 2 9 6 1
389-ds-base 2.4.5-23.el... Red Hat type=cyclonedx Nov 25, 2025 755 5 1 3 1
389-ds-base 1.4.3.39-23... Red Hat type=cyclonedx Apr 07, 2026 2079 18 2 9 6 1
389-ds-base 1.4.2.4-19.... Red Hat type=cyclonedx Apr 07, 2026 530 80 9 41 28 1 1
389-ds-base 2.4.5-24.el... Red Hat type=cyclonedx Apr 07, 2026 755 5 1 3 1
389-ds-base 2.6.1-20.el9... Red Hat type=cyclonedx Apr 07, 2026 2581 6 1 3 2
://localhost:8080


TRUSTIFICATION
This instance is running in read-only mode. Uploads, imports, and other modifications are disabled.
Dashboard
Search
SBOMs
All SBOMs
Groups
Vulnerabilities
Packages
Licenses
Cryptography
Models
Advisories
Importers
API
openjdk-els/openjdk-11-runtime-rhel8
cyclonedx
Actions
Info
Packages
Vulnerabilities
Models
Cryptography
3
Total vulnerabilities
Critical: 0
High: 3
Medium: 0
Low: 0
None: 0
Unknown: 0
Name
openjdk-els/openjdk-11-runtime-rhel8
Version
Creation date
Dec 18, 2025
Show VEX resolutions
1 - 3 of 3
1
of 1
Id
Description
CVSS
Affected...
Remediations
Published
Updated
CVE-2022-40897
pypa/setuptools vulnerable to Regular E
High (7.5)
1 Sources
1
0 Remediations
Dec 22, 2022
Oct 29, 2024
CVE-2024-63
setuptools vulnerable to Command Inje
High (8.8)
1 Sources
1
0 Remediations
Jul 15, 2024
Aug 01, 2024

Trustify has to manage much more than just SBOMs



because we need Vulnerability correlation



Version in range ?

SBOMs are boring

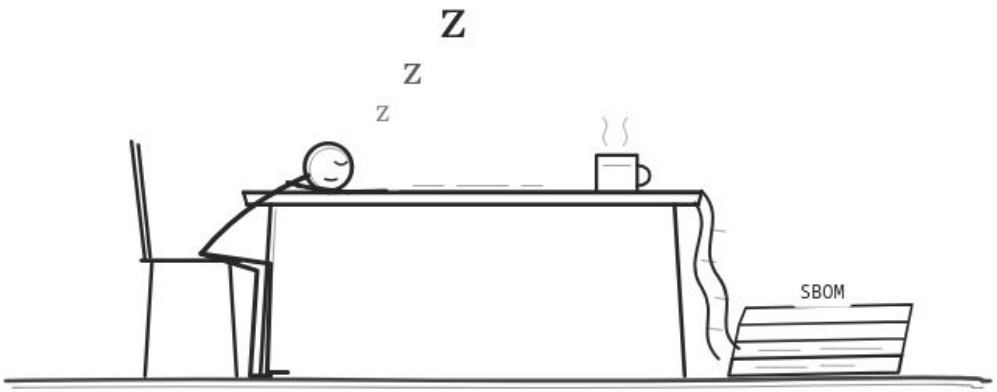
Manifest of software components

Transitive dependencies

Other important relationships:

- upstream
- src -> binary
- image index container -> child container
- product ->

Provide useful meta data (license, crypto)



sboms are boring

SBOMs in 2026: Everyone's generating them, no one's using them





https://www.enisa.europa.eu/sites/default/files/2026-06/SBOM%20Adoption%20State%20of%20Play%202026.pdf

ENISA's *SBOM Adoption State of Play 2026* (June 2026) reports a survey of 334 organisations



- **78%** have started using SBOMs, but most are still piloting (44%). Only 25% report broad adoption and 9% a fully automated, mature setup.
- **42%** of organisations mostly see SBOMs as a "defensive necessity" and 28% call them a compliance burden.
- **43%** say CRA significantly accelerated their tooling investment.
- **62%** rate SBOM completeness as adoption barrier.
- **58%** rate vulnerability matching an adoption barrier: aligning CPE and PURL identifiers, and false positives.
- **~55%** don't know how their organisation consumes the SBOMs it receives.

https://www.enisa.europa.eu/sites/default/files/2026-06/SBOM%20Adoption%20State%20of%20Play%202026.pdf

arXiv Search Submit Donate Log in

Computer Science > Software Engineering

[Submitted on 6 Jul 2026]

Beyond Compli
GitHub SBOMs

Kawsar Ahmed Bhuiyar

Modern software develop
chain attacks exploiting k
their versions, and their p
providing valuable metac
analysis, and how incom
10,000 GitHub repositori
Trivy, and the Microsoft S
find that component vers
GitHub yields results sim
discuss potential shortco

Subjects: Software Engineeri
Cite as: arXiv:2607.04614 [cs
(or arXiv:2607.04614
https://doi.org/10.485

Submission history

From: Diego Elias Costa [viev
[v1] Mon, 6 Jul 2026 02:44:5

Access Paper:

ew PDF
TML (experimental)
X Source
nse

nt browse context:

v | next >
ecent | 2026-07
e to browse by:

ences & Citations

SA ADS
ogle Scholar
mantic Scholar

"Beyond Compliance: A Large Scale Study on the Completeness and Consistency of the GitHub SBOMs" by Bhuiyan, Besbes, Raj, Al Assil and Costa (REALISE Lab, Concordia University), posted to arXiv on 6 July 2026

- **No GitHub SBOM is NTIA-compliant.** Supplier information is missing from every SBOM in every language. Name, version, unique ID, relationships, author and timestamp are always present.
- **Version coverage depends heavily on the ecosystem.** For top-level dependencies, Swift reaches 98.8% and JavaScript 96.2%, but Java gets only 41.4% and Python 29.9%.
- **The 100% transitive version coverage is an artifact.** GitHub drops the transitive dependencies of any parent that has no version, so the missing data is hidden rather than resolved.
- **More components doesn't mean more accurate.** GitHub reports the most components and finds the most vulnerabilities. Against ground truth, though, the Micros clearly on Python (F1 47.9% vs 29.0%) and Java (84.6% vs 35.9%). GitHu JavaScript (F1 85.3%), most likely because JavaScript projects have lockf versions.
- **PURL coverage varies widely.** GitHub gives a PURL for 100% of components and Microsoft for nearly all. Syft covers 60–90%, while Trivy drops as low as 17.7% for C.
- **Missing versions inflate vulnerability results.** In their setuptools example, a versione PURL matched 2 vulnerabilities and a versionless one matched 6.
- **The tools disagree, especially on versions.** For Go, GitHub and Microsoft agree on 76.8% of components but only 32.6% once versions are compared.

At Red Hat we audited ~750k internal build sboms, only 27% were clean, 65% had warnings



SBOMS are not boring

Different ‘kinds’ of sboms https://www.cisa.gov/sites/default/files/2023-04/sbom-types-document-508c.pdf

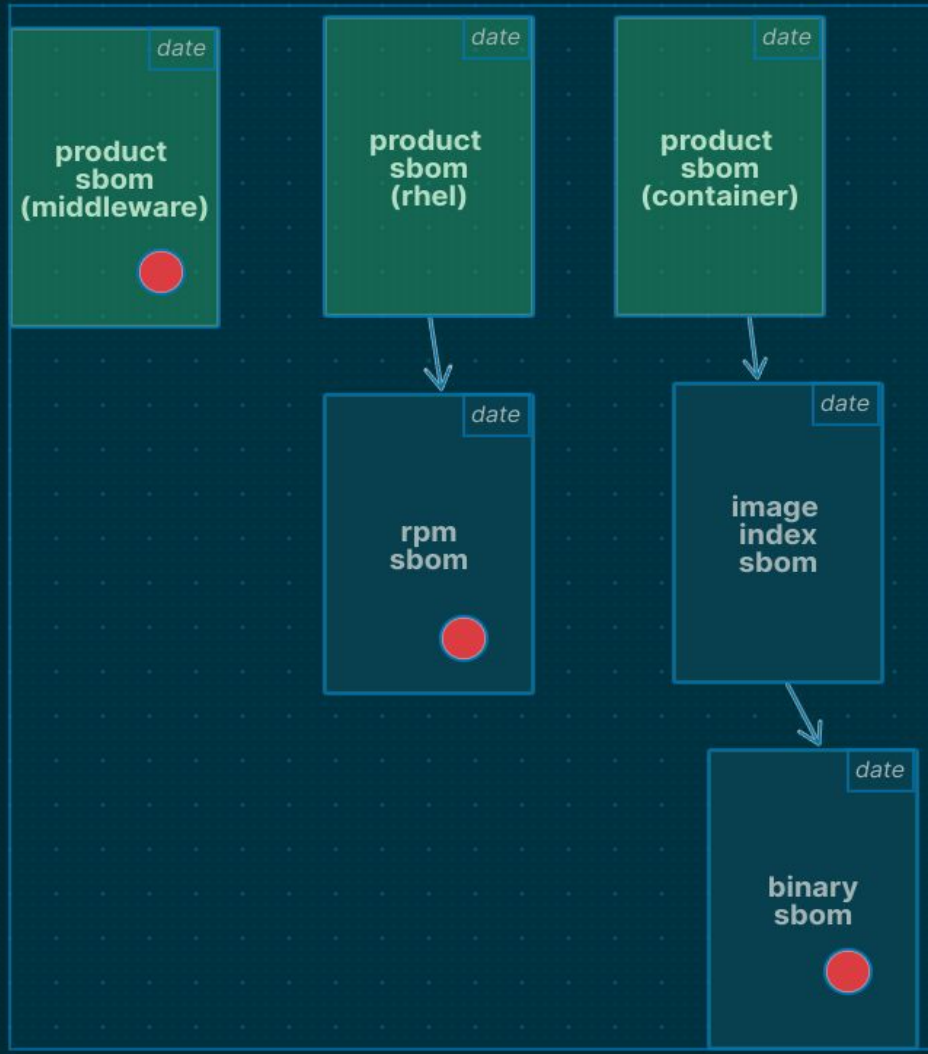
- src sbom
- build sbom
- release sbom
- product sbom
- runtime/deploy/instance sboms (generated by syft/trivy)
- container sbom

Say different things ….

Shallow vs complete SBOMs

one giant sbom (redhat)

linking sboms (chainguard)



At Red Hat we have all the SBOM distributions.



match on 'openssl'

filter out latest and load into graphmap cache

OLDER

LATEST

OLDER

LATEST

date

product sbom (rhel)

date

product sbom (rhel)

date

product sbom (container)

date

product sbom (container)

graphmap cache

date

rpm sbom

date

rpm sbom

date

image index sbom

date

image index sbom

date

binary sbom

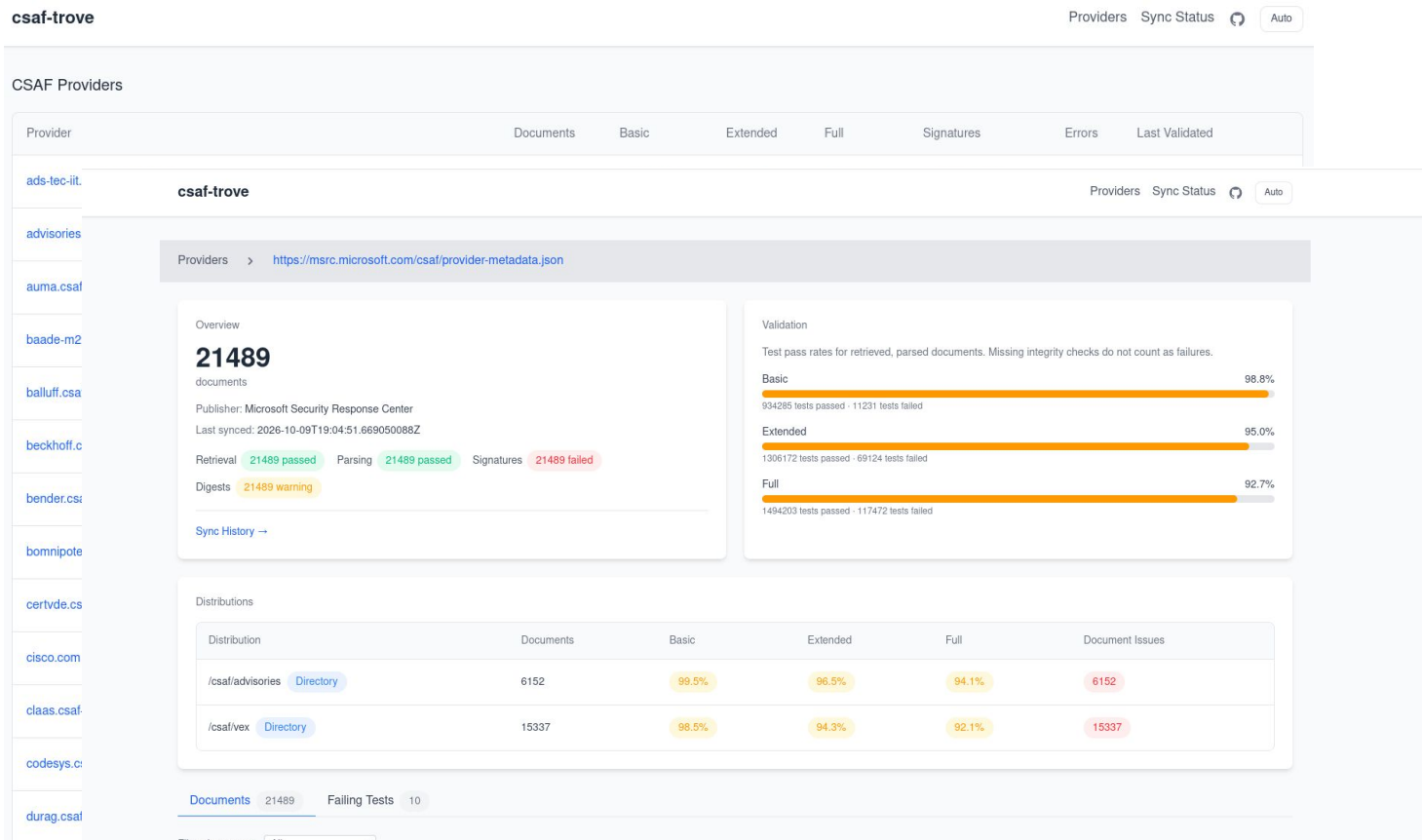
date

binary sbom

SBOM external linking creates performance at scale challenges.

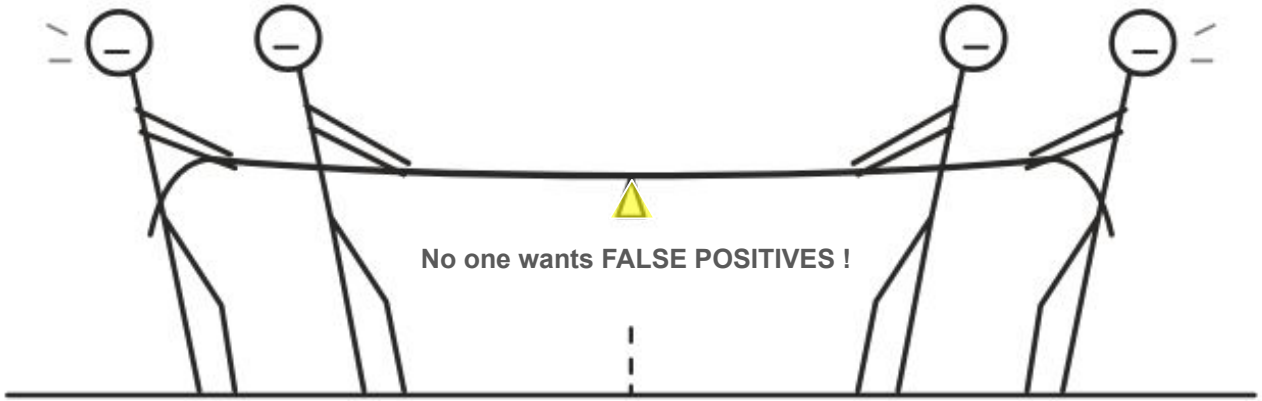


Advisory quality - https://csaf.dentrassi.de



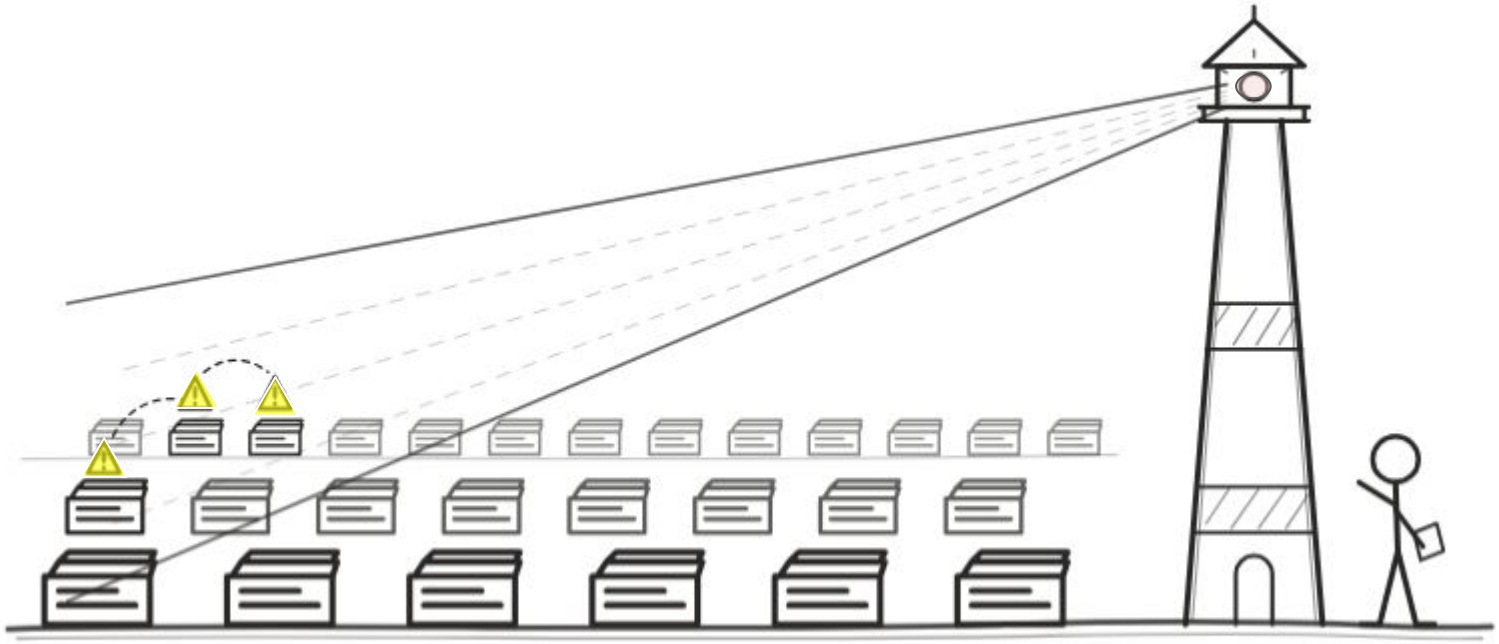
Developers want to know minimal fix

Users want to know maximum remediation

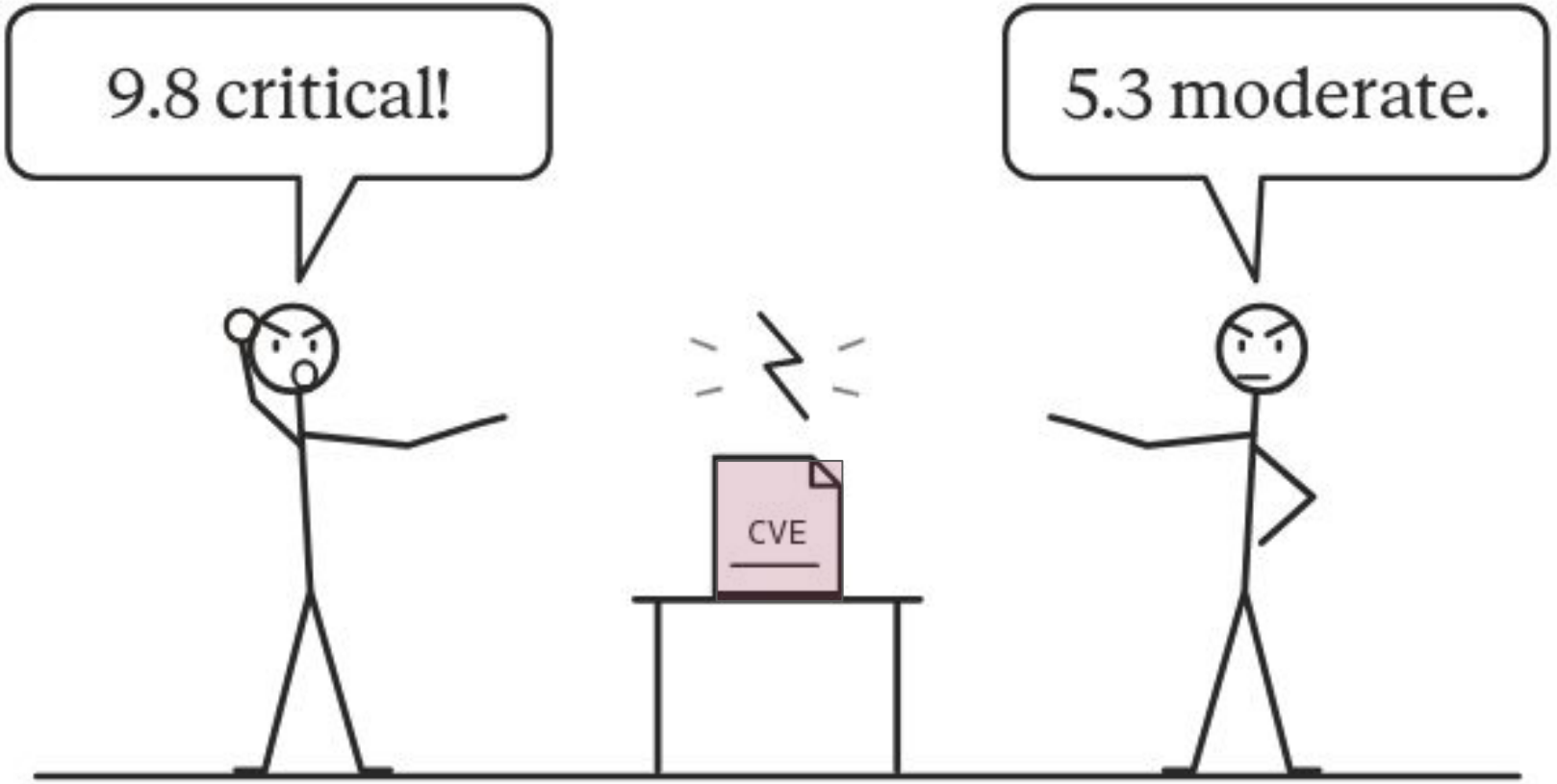


No one especially wants FALSE NEGATIVES !

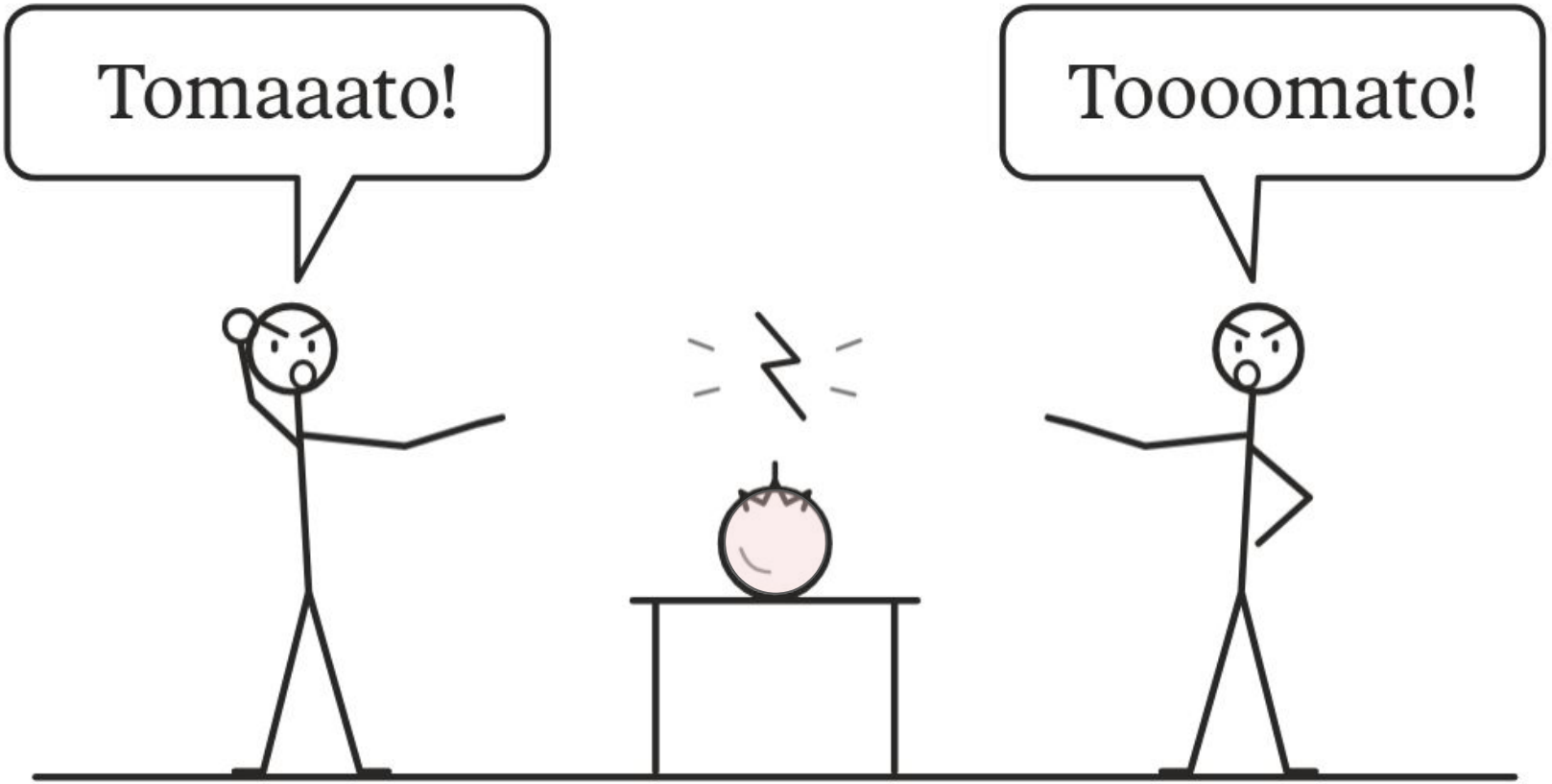
Demo trustify II



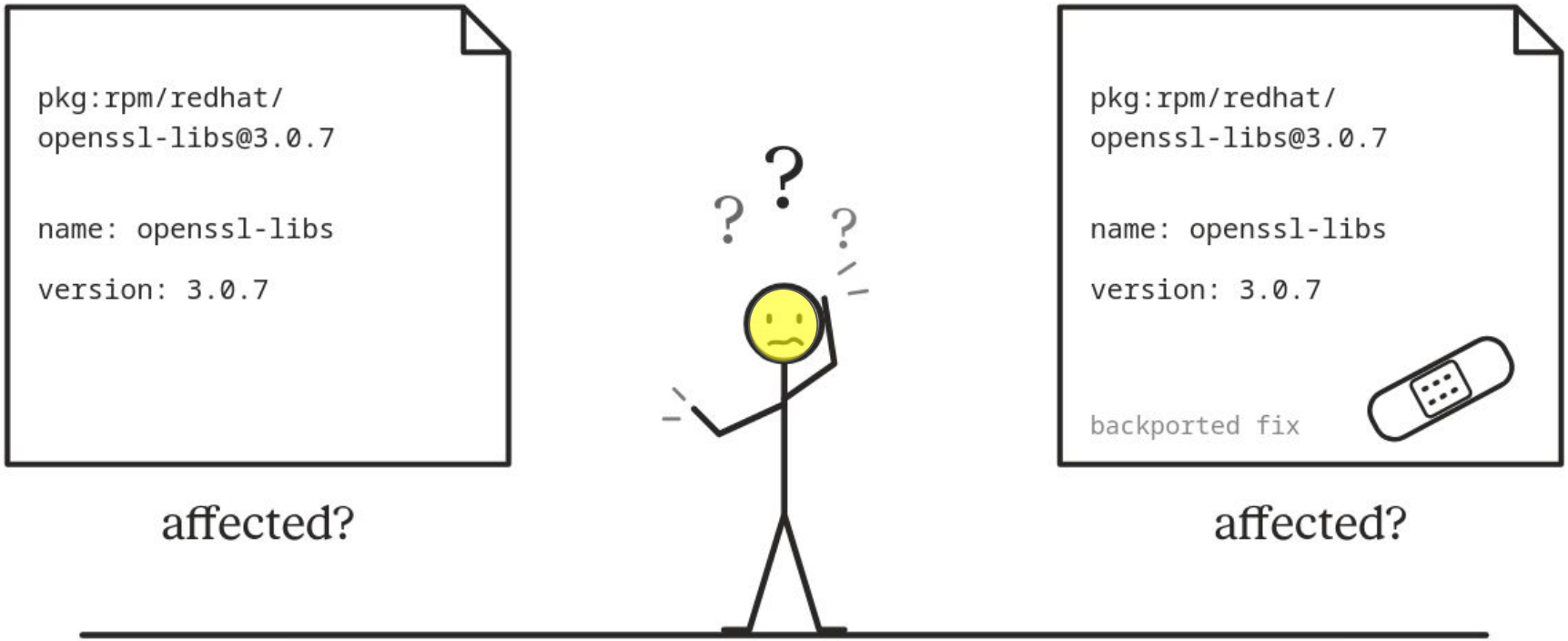
Experts don’t always agree



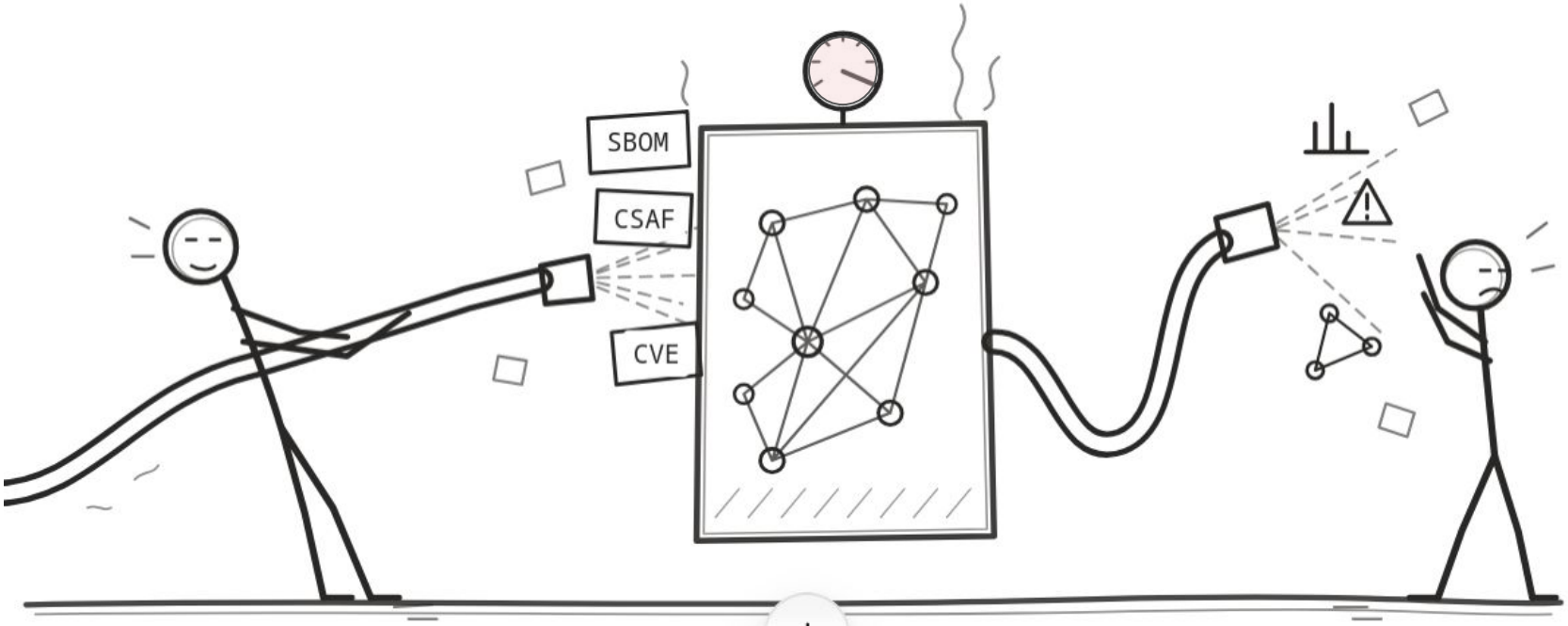
Identity confusion



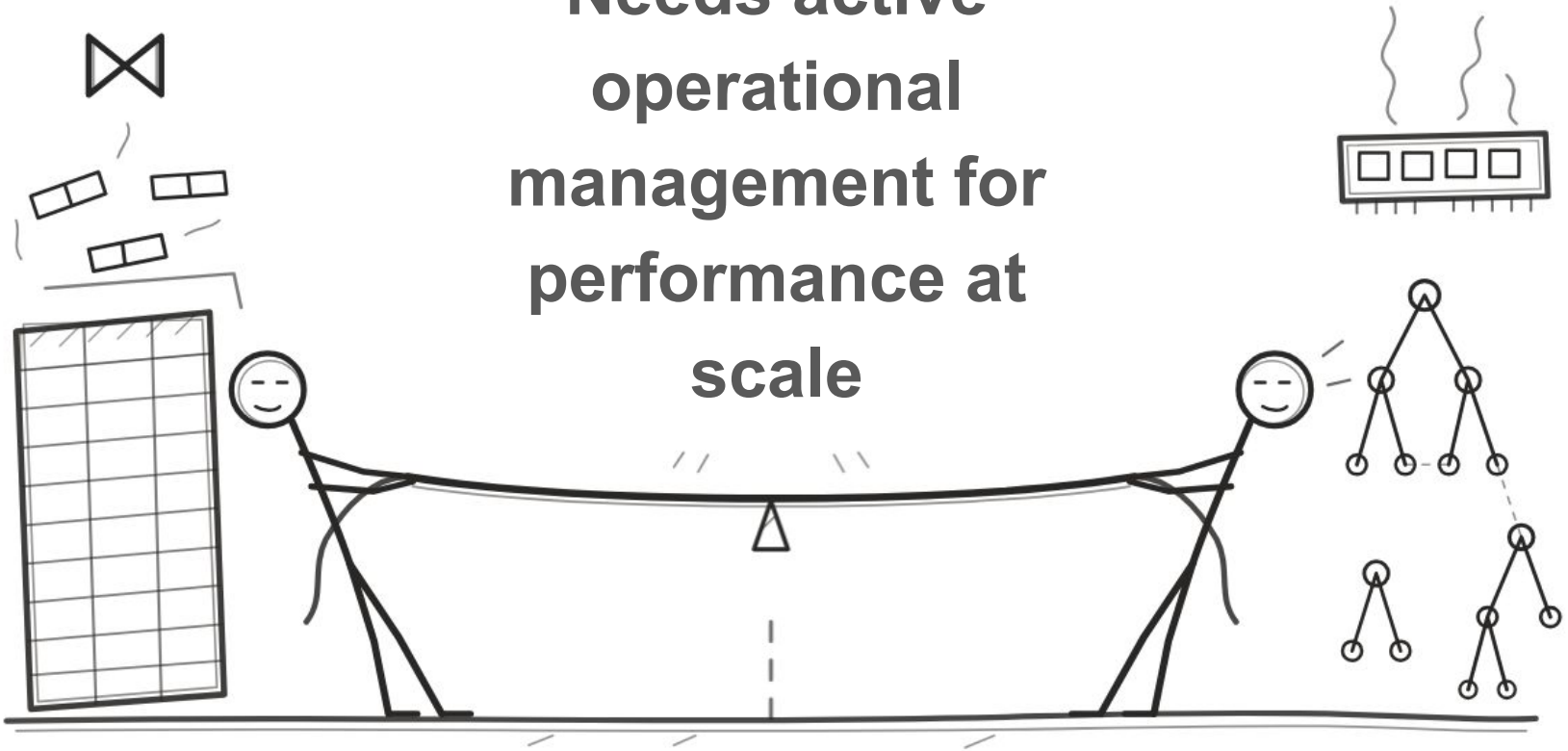
Backports


pkg:rpm/redhat/
openssl-libs@3.0.7
name: openssl-libs
version: 3.0.7
affected?
pkg:rpm/redhat/
openssl-libs@3.0.7
name: openssl-libs
version: 3.0.7
backported fix
affected?

‘Double firehose’ problem



Needs active operational management for performance at scale



Trustify load tests

Target **trustify (scale snapshot, CI)** · Latest run **2026-10-11 03:43 UTC**

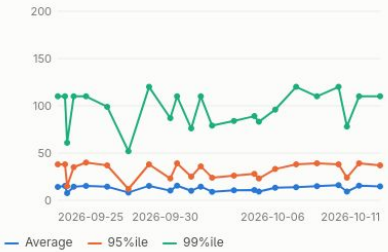
Test runs	Latest avg response	Latest 95%ile	Latest throughput	Latest failure rate	Endpoints tested
23 2026-09-25 → 2026-10-11	**14.8** ms (-0.7)	**37** ms (-2)	**625.7** req/s (+28.0)	**0.0%** 0 of 375,070 requests	**72** 72 across all runs

Trends

Time range: All time · Endpoint: All endpoints (aggregated) · 23 of 23 runs

Response time

Per run, ms · click a point to open that run



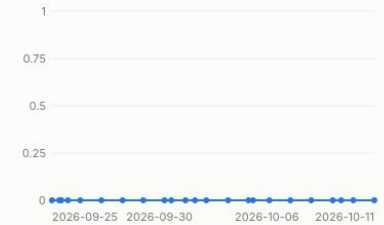
Throughput

Requests per second, per run



Failure rate

Failed requests, % of all requests



Latest run vs previous

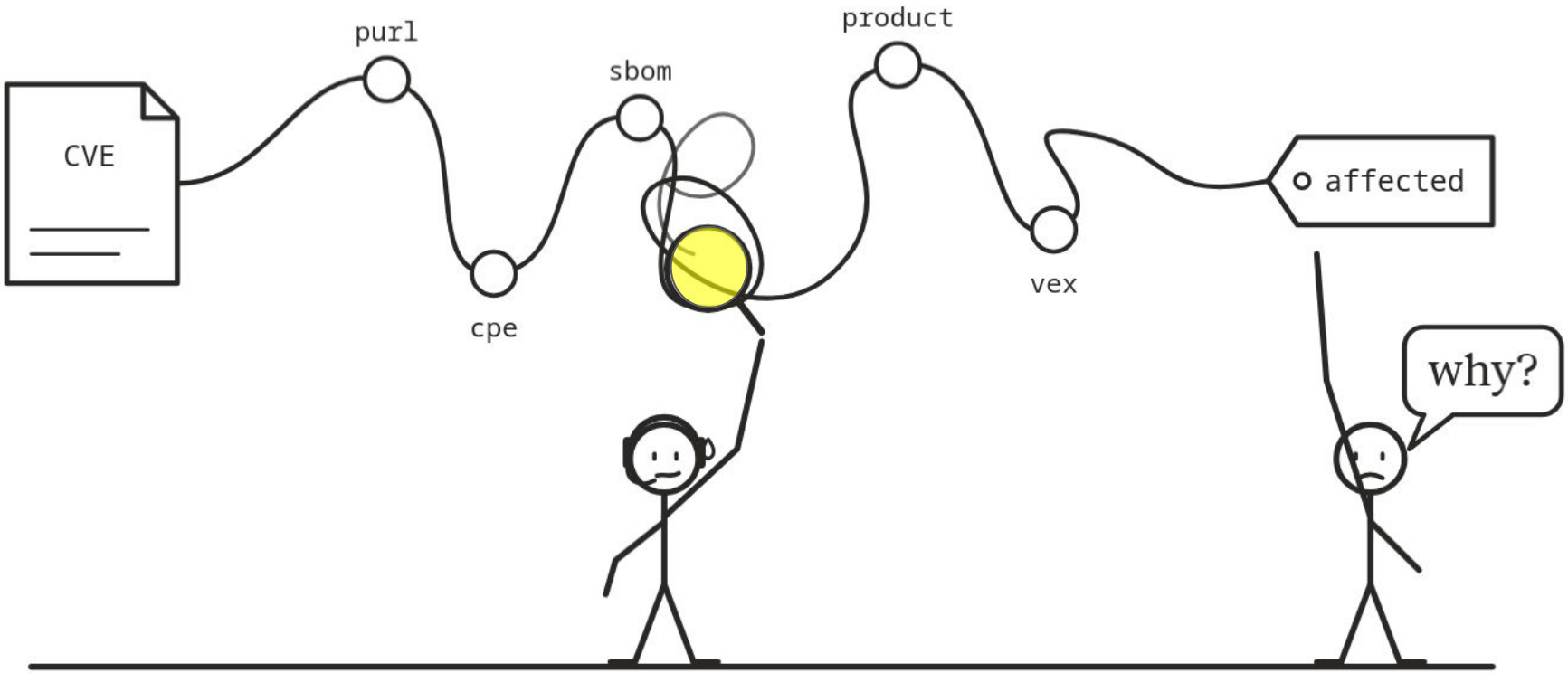
Comparing 2026-10-11T02-38-14_af6ffd7 with 2026-10-10T02-35-33_af6ffd7. Flagged when 95%ile up or down by at least 10% and 5 ms.

Regressions 14

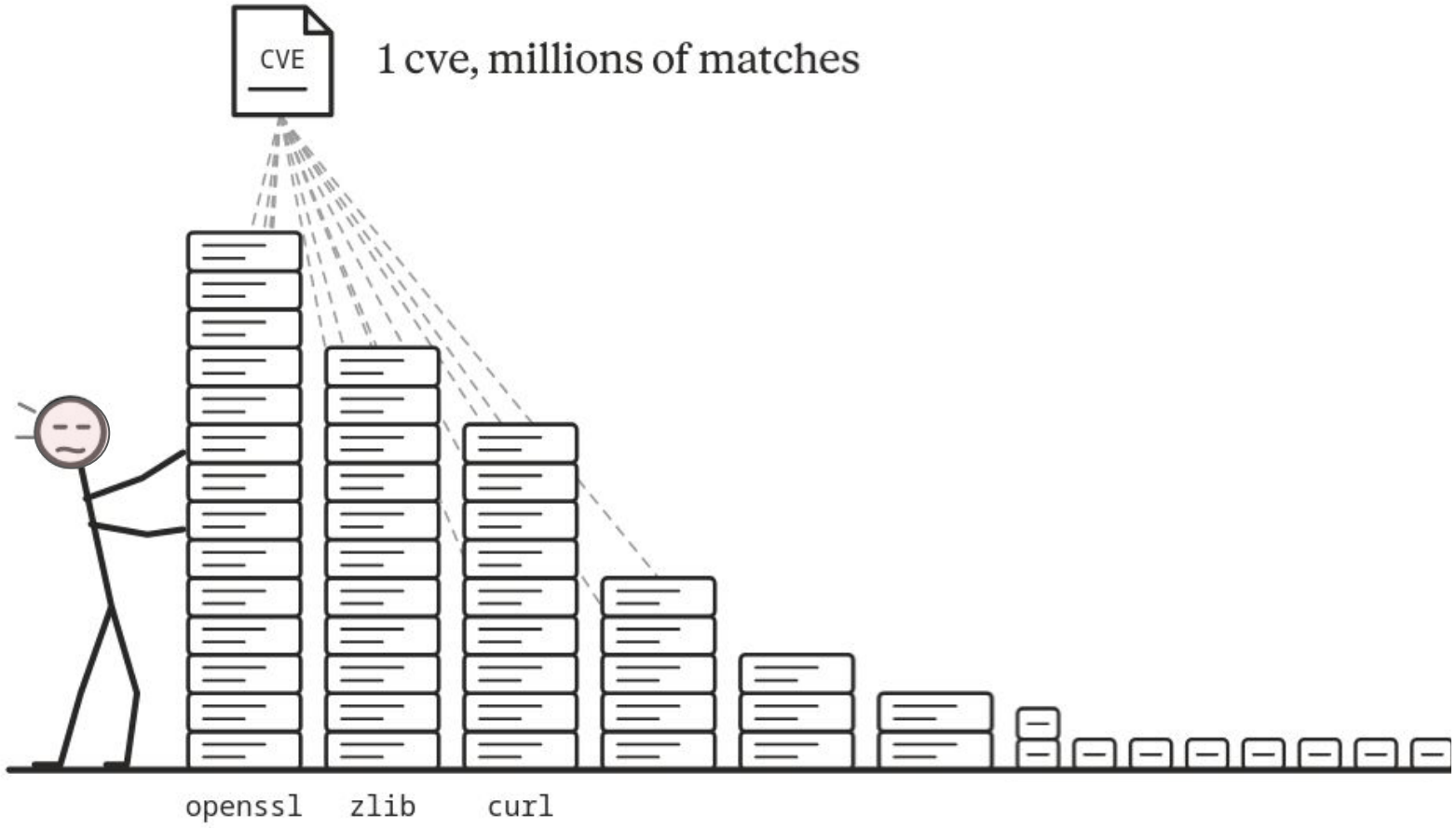
Endpoint	Prev 95%ile	Latest 95%ile	Δ ms	Δ %	Prev avg	Latest avg	Fails
GET /api/v3/purl?sort=purl:name:asc&total=true	38	74	(+36)	+95%	109	85.7	0 → 0
GET /api/v3/purl?q=license~GPLv3+...&sort=name:desc&total=true	66	89	(+23)	+35%	28.3	32.0	0 → 0
GET /api/v3/sbom/count-by-package	49	66	(+17)	+35%	22.2	25.8	0 → 0
GET /api/v3/purl?q=name=curl&total=true	37	49	(+12)	+32%	14.5	16.8	0 → 0
GET /api/v3/purl/base?q=namespace=redhat&total=true	73	95	(+22)	+30%	43.7	44.6	0 → 0
GET /api/v3/purl?q=purl:ty=rpm&total=true	39	50	(+11)	+28%	108	87.9	0 → 0
GET /api/v3/vulnerability?q=base_score>=7.0&total=true	210	260	(+50)	+24%	34.3	40.8	0 → 0

https://guacsec.github.io/trustify-scale-test-runs/perf/

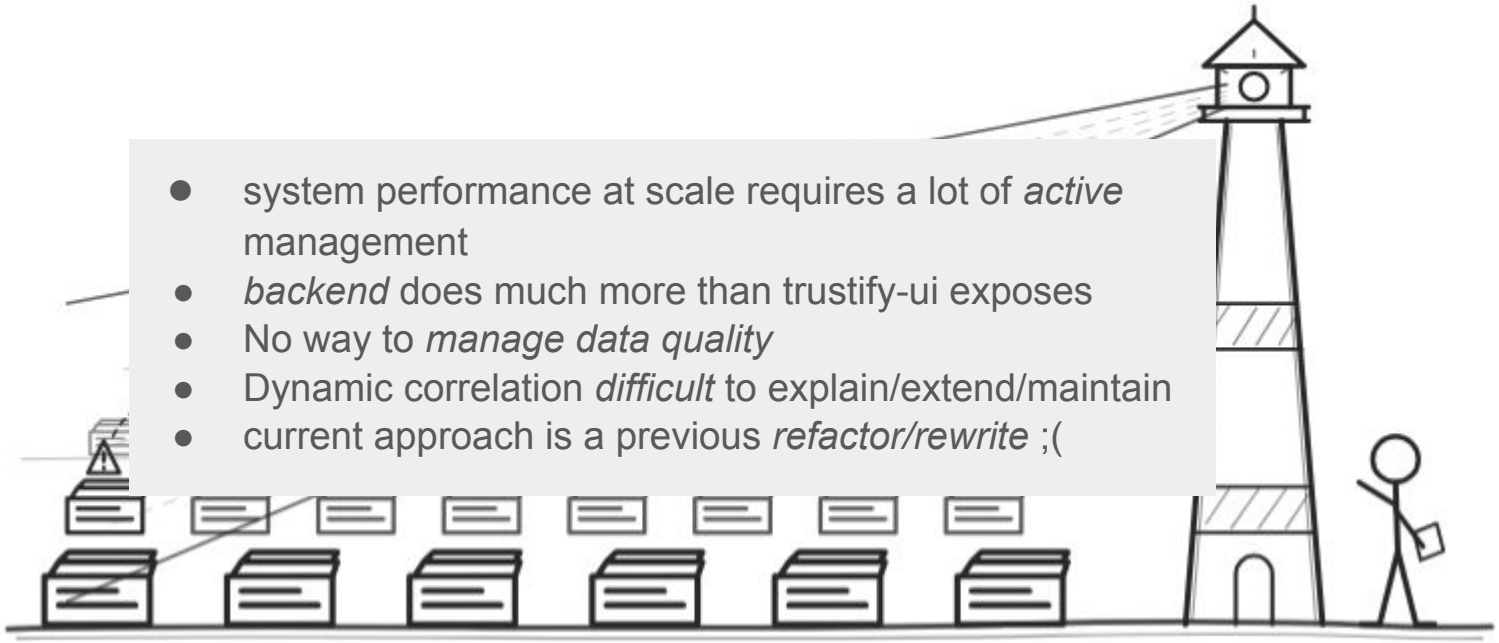
Explaining/debug correlation is hard



Unbalanced data



Scale challenges



- system performance at scale requires a lot of *active* management
- *backend* does much more than trustify-ui exposes
- No way to *manage data quality*
- Dynamic correlation *difficult* to explain/extend/maintain
- current approach is a previous *refactor/rewrite* ;(

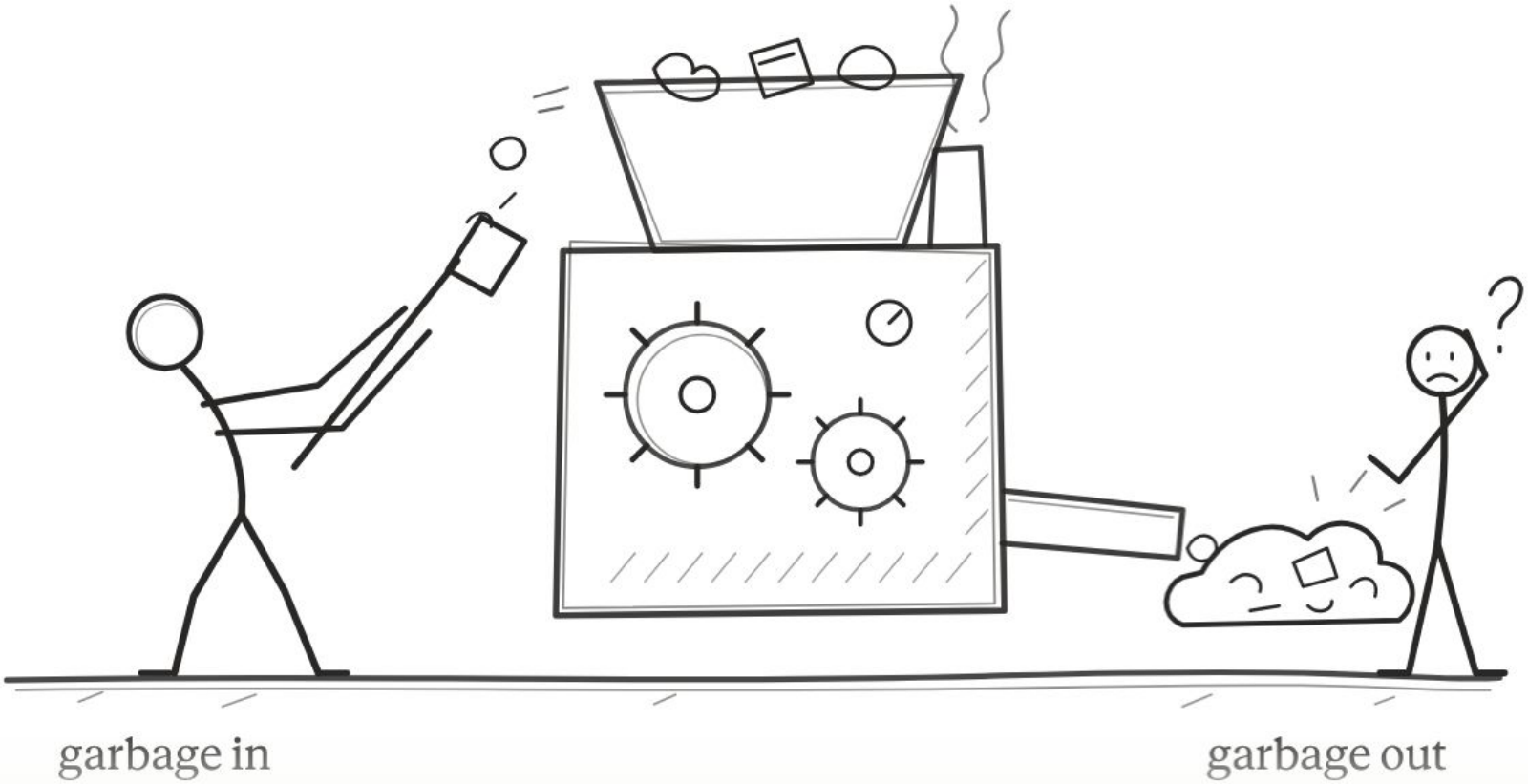

You are not
SBOMing correctly !


You need to use
more SBOMs !!!!

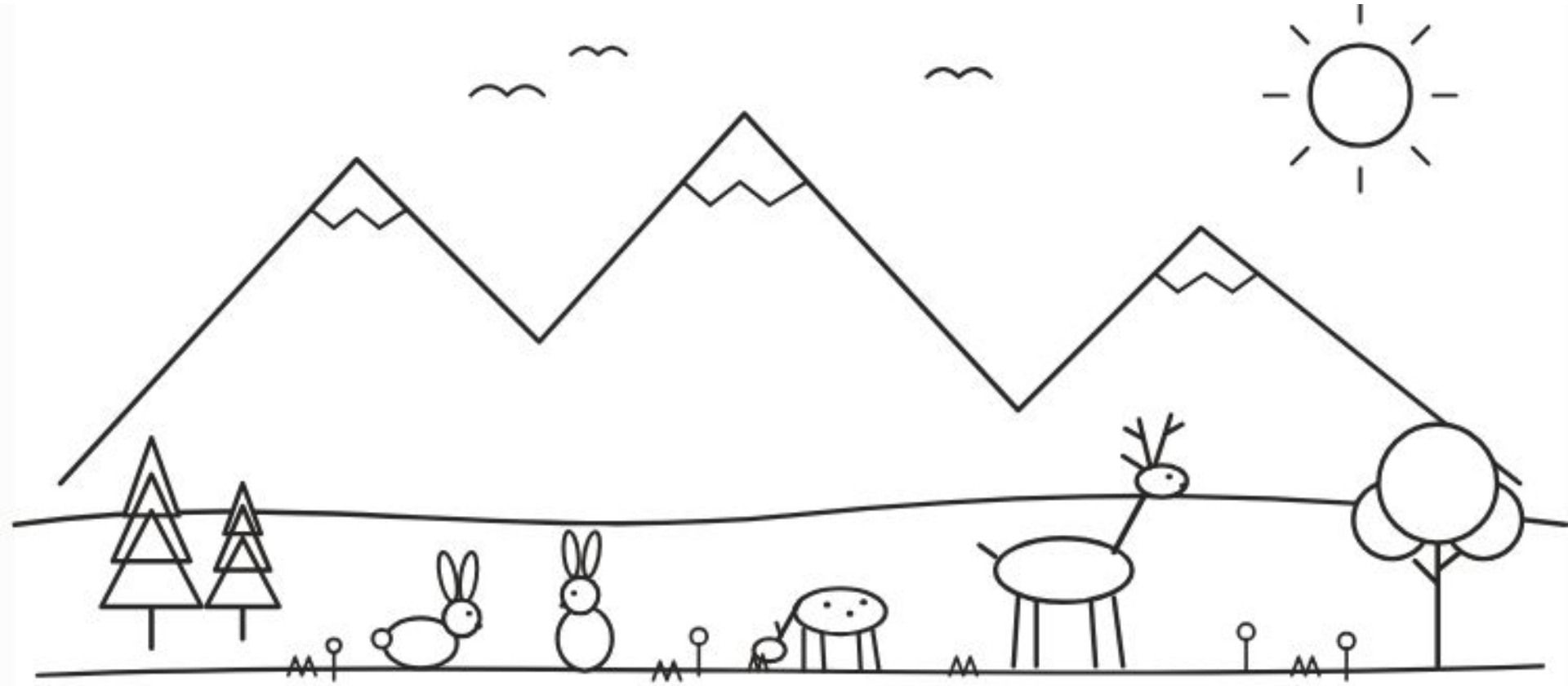

You are not
SBOMing hard
enough !!!!


You need to use the
right SBOM tool !

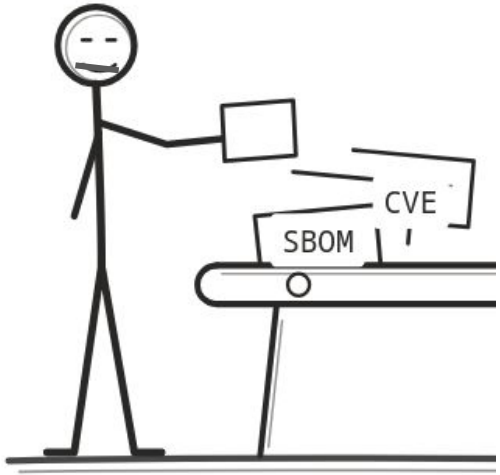
GIGO



Interval

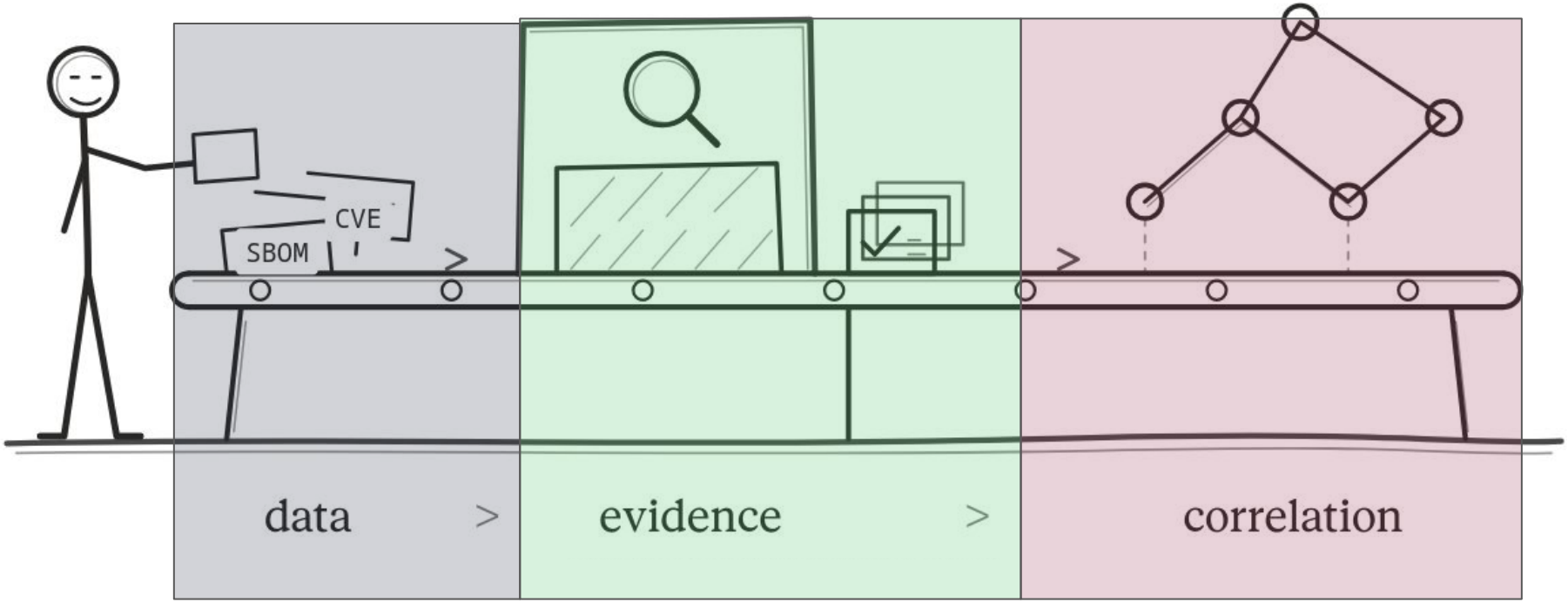


Are we happy with dynamic correlation ?

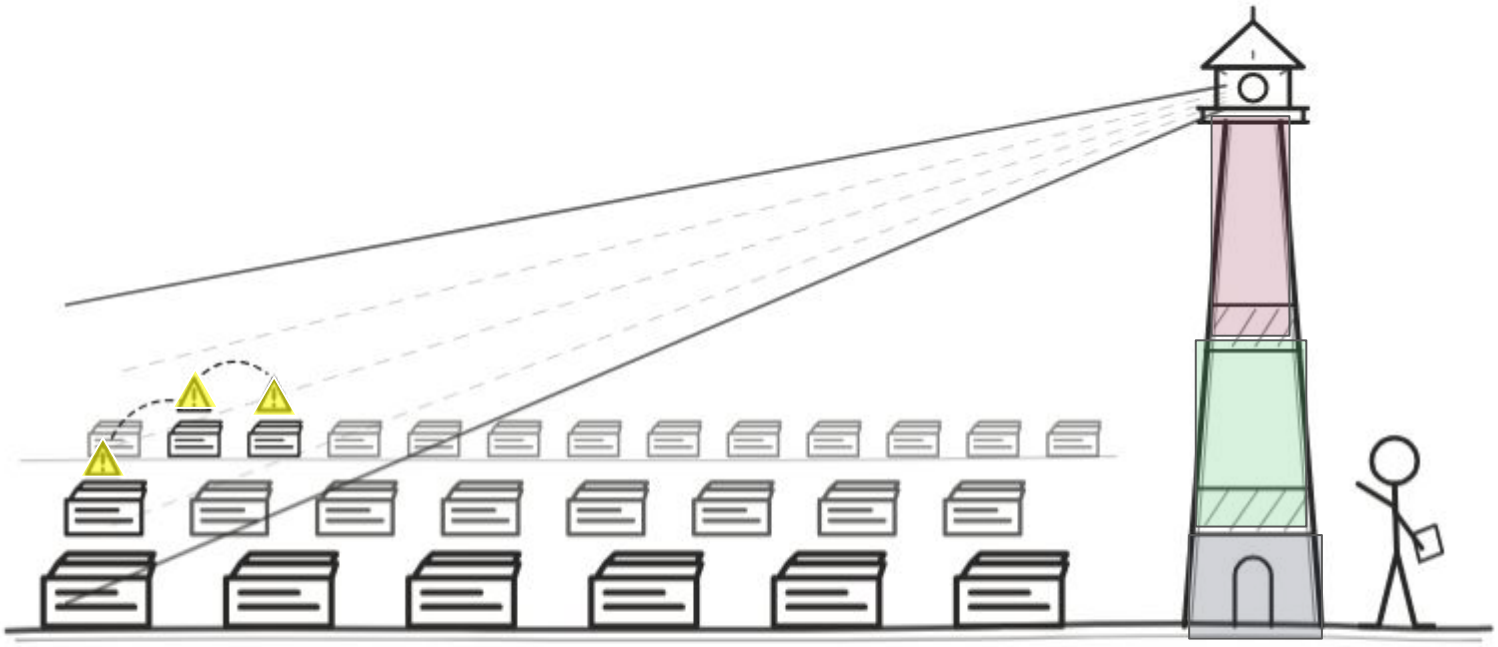


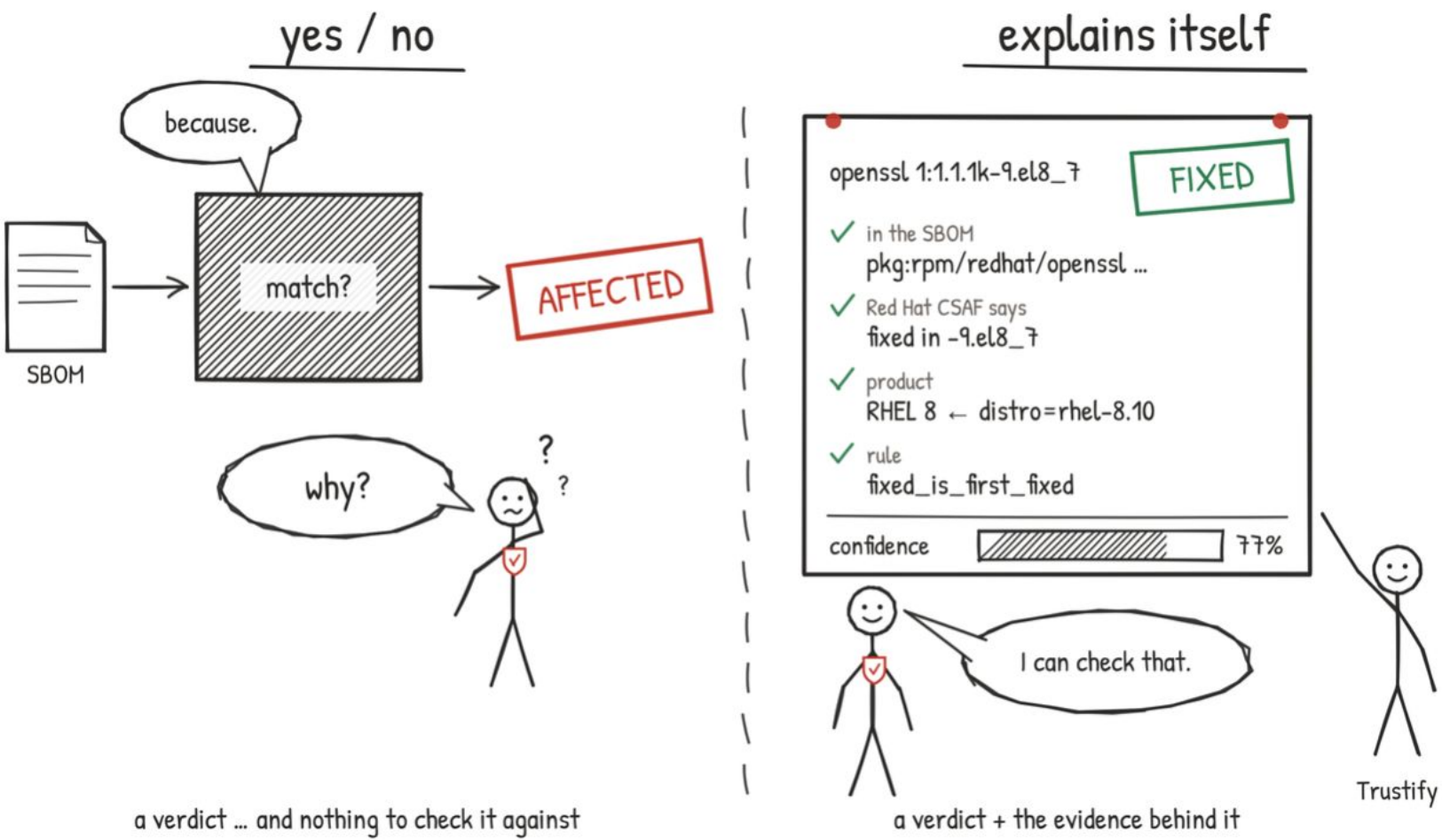
NO

Deterministic pipeline

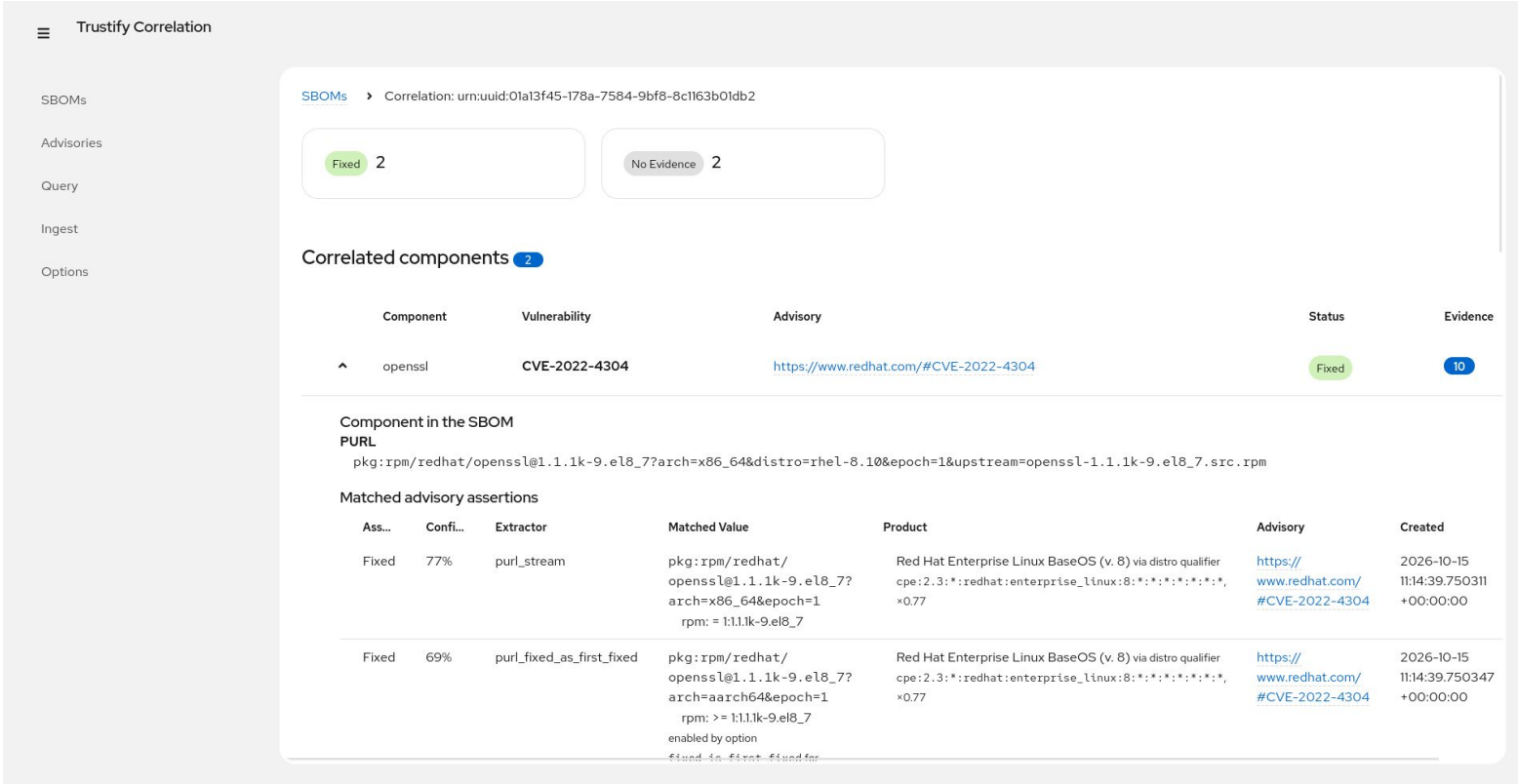


Demo trustify III




yes / no
because.
match?
SBOM
AFFECTED
why?
?
?
a verdict … and nothing to check it against
explains itself
openssl 1:1.1.1k-9.el8_7
FIXED
in the SBOM
pkg:rpm/redhat/openssl …
Red Hat CSAF says
fixed in -9.el8_7
product
RHEL 8 ← distro=rhel-8.10
rule
fixed_is_first_fixed
confidence
77%
I can check that.
Trustify
a verdict + the evidence behind it

Correlation with provenance/evidence not booleans



Why there are 10 evidence rows per CVE: five architectures × two rules.

- The Red Hat CSAF lists the fixing build once per architecture (`x86_64`, `aarch64`, `ppc64le`, `s390x`, `src`).
- Every one of those matched, because the PURL extractor doesn't filter on the `arch` qualifier.
- Each architecture is matched by two rules:

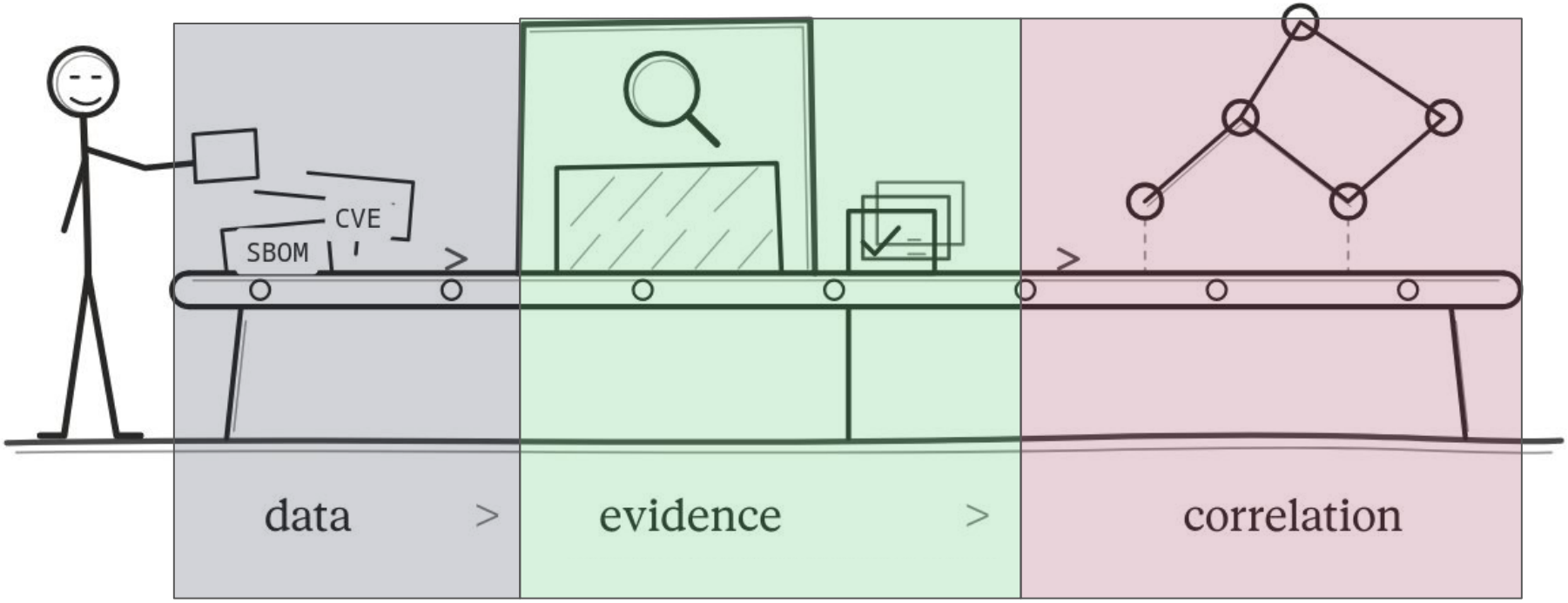
Extractor	Range	Confidence	Meaning
`purl_stream`	`= 1:1.1.1k-9.el8_7`	77%	Exact match: this is the fixing build
`purl_fixed_as_first_fixed`	`>= 1:1.1.1k-9.el8_7`	69%	Derived: at or after the fix means fixed. Enabled by the Red Hat `fixed_is_first_fixed` option, and the UI says so on the row.

Where 77% and 69% come from:

- **Context factor, 0.85 × 0.9 = 0.77.** The product context was inferred from the PURL's `distro=rhel-8.10` qualifier (0.85). It then matched `enterprise_linux:8` through an ANY CPE part, `cpe:2.3:*:…` (0.9).
- **Rule factor.** The exact rule counts as 1.0, giving 77%. The derived rule counts as 0.9, giving 0.77 × 0.9 ≈ 69%.

Correlation

Point in time

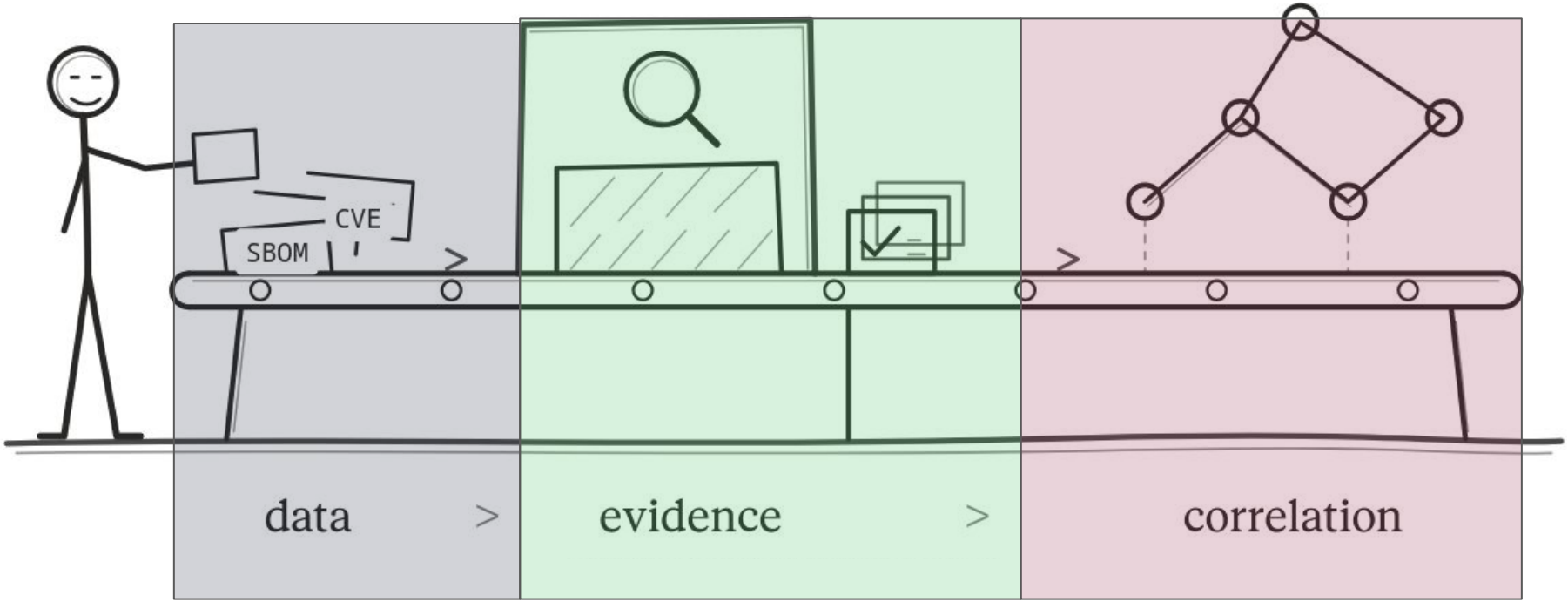


Extract evidence at ingestion time

Read single table for correlation

validators

score data



gate data

scheck

Semantic validation of structured data. Define rules in JSON, DSL, Schematron XML, or plain English. Paths use JSONPath (RFC 9535). Inspired by ISO Schematron.

Rules CycloneDX Quality + NTIA Example: Basic fields

JSON | DSL | Schematron | Free-text

```
{
  "title": "CycloneDX SBOM Quality",
  "description": "Quality and completeness checks for CycloneDX SBOMs, derived from sbomqs, sbom-scorecard, and NTIA minimum element criteria. Complements cyclonedx-min.json with deeper component-level quality checks.",
  "default_phase": "ntia",
  "phases": [
    {
      "name": "ntia",
      "active_patterns": [
        "ntia-sbom-fields",
        "ntia-component-fields"
      ]
    },
    {
      "name": "quality"
```

Document Load file

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.5",
  "version": 1,
  "serialNumber": "urn:uuid:550e8400-e29b-41d4-a716-446655440000",
  "metadata": {
    "timestamp": "2024-01-15T10:00:00Z",
    "tools": [{ "name": "scheck-test", "version": "1.0.0" }],
    "authors": [{ "name": "Test Author" }],
    "component": {
      "type": "application",
      "name": "my-app",
      "version": "1.0.0"
    }
  },
  "components": [
    {
      "type": "library",
      "name": "serde"
```

Validate | JSON report

Output

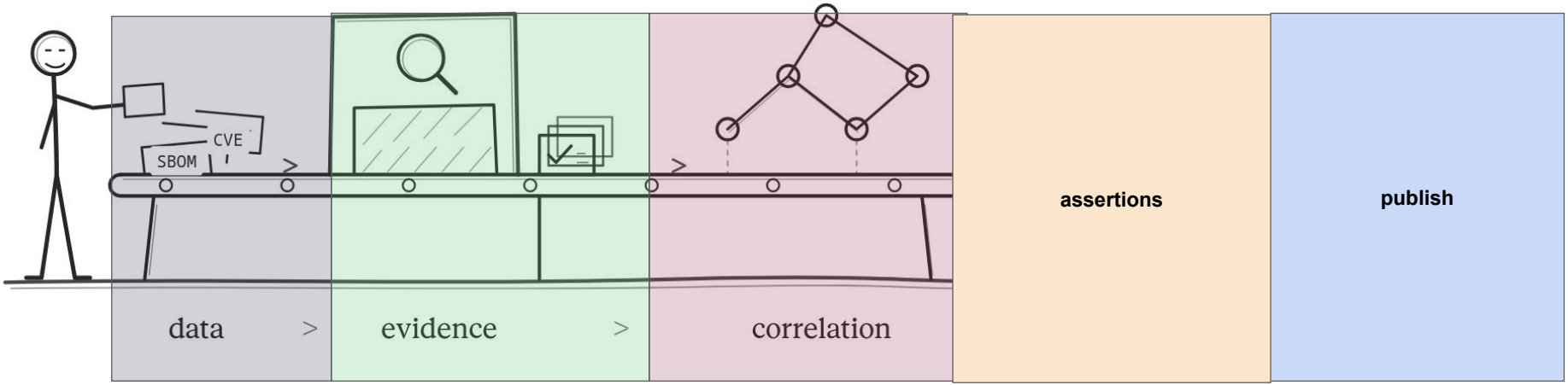
```
    {
      "rule": "",
      "pattern": "ntia-component-fields",
      "context": "$['components'][0]"
    }
  ],
  "findings": [
  ],
  "summary": {
    "fatal": 0,
    "errors": 0,
    "warnings": 0,
    "infos": 0
  }
}
```

▸ CLI Usage

https://rh-ifuller.github.io/scheck

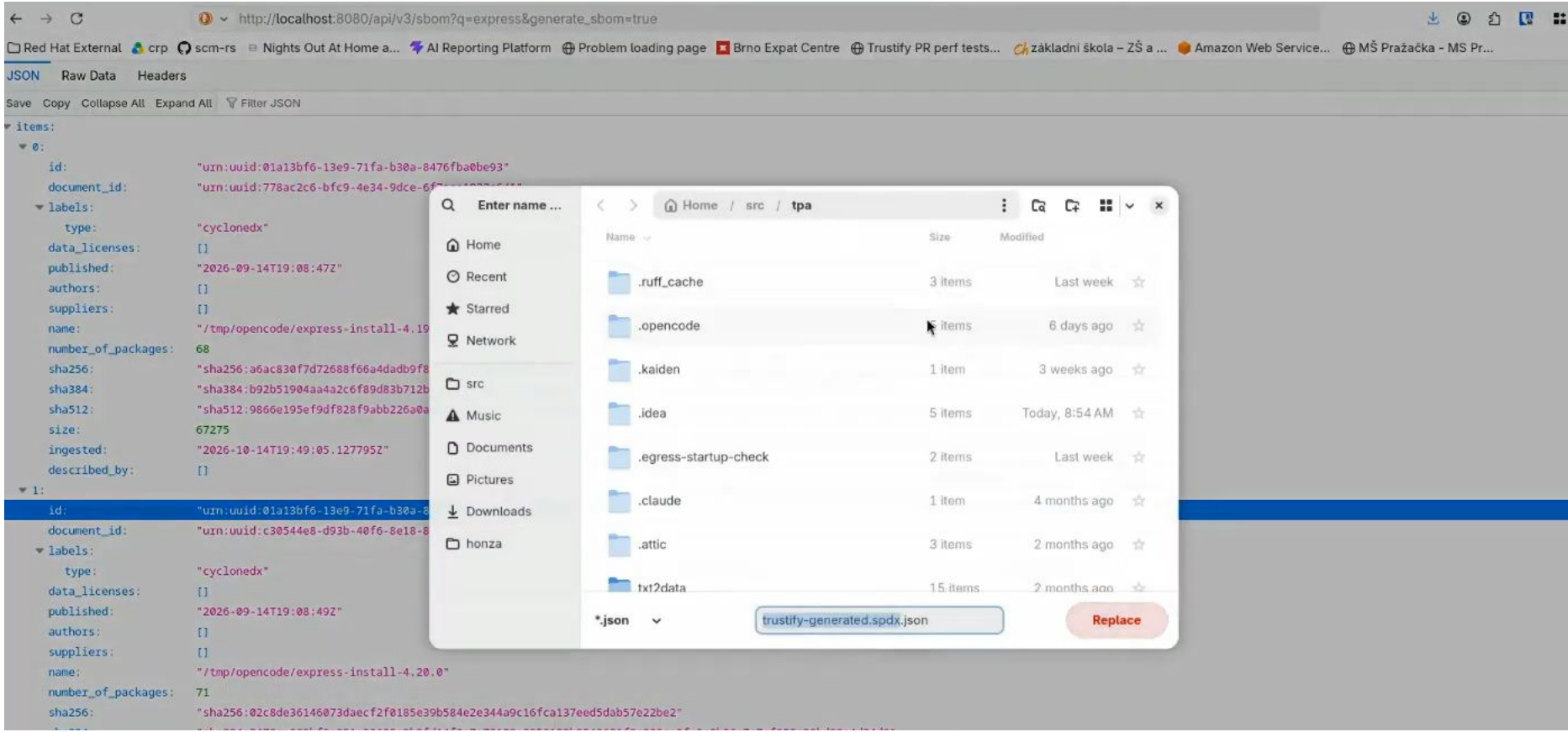
Developers want to publish sboms

(and CSAF vex …)



Generate aggregated sbom

Generate sboms



Generate VEX

CVE-2025-48924	Apache Commons Lang, Apache Commons Lang: ClassUtils.getClass(...) can throw a S...	medium (5.3)	10	Ready	7/11/2025

2. Affected SBOMs

10 affected SBOM(s) — 0 selected for VEX statement

SBOM Name	Version	Affected Packages
rhn_satellite_6.17	-	1 package(s)
ocp-tools-4.13	-	2 package(s)
examples	4.8.0.redhat-00001	1 package(s)
ocp-tools-4.12	-	2 package(s)
quarkus-bom-3.8.3.redhat-00003	3.8.3.redhat-00003	1 package(s)
quarkus-bom	2.13.8.Final-redhat-00004	1 package(s)
jboss-eap-7/eap74-els-openjdk17-openshift-rhel8	-	2 package(s)
ocp-tools-4.14	-	2 package(s)
demo	0.0.1-SNAPSHOT	1 package(s)
rhbk-26.2	-	2 package(s)

3. VEX Statement

Generate VEX

☑	examples	4.8.0.redhat-00001	1 package(s)
☐	ocp-tools-4.12	-	2 package(s)
☐	quarkus-bom-3.8.3.redhat-00003	3.8.3.redhat-00003	1 package(s)
☐	quarkus-bom	2.13.8.Final-redhat-00004	1 package(s)
☐	jboss-eap-7/eap74-els-openjdk17-openshift-rhel8	-	2 package(s)
☐	ocp-tools-4.14	-	2 package(s)
☐	demo	0.0.1-SNAPSHOT	1 package(s)
☐	rhbk-26.2	-	2 package(s)

3. VEX Statement

Product Status *

Fixed

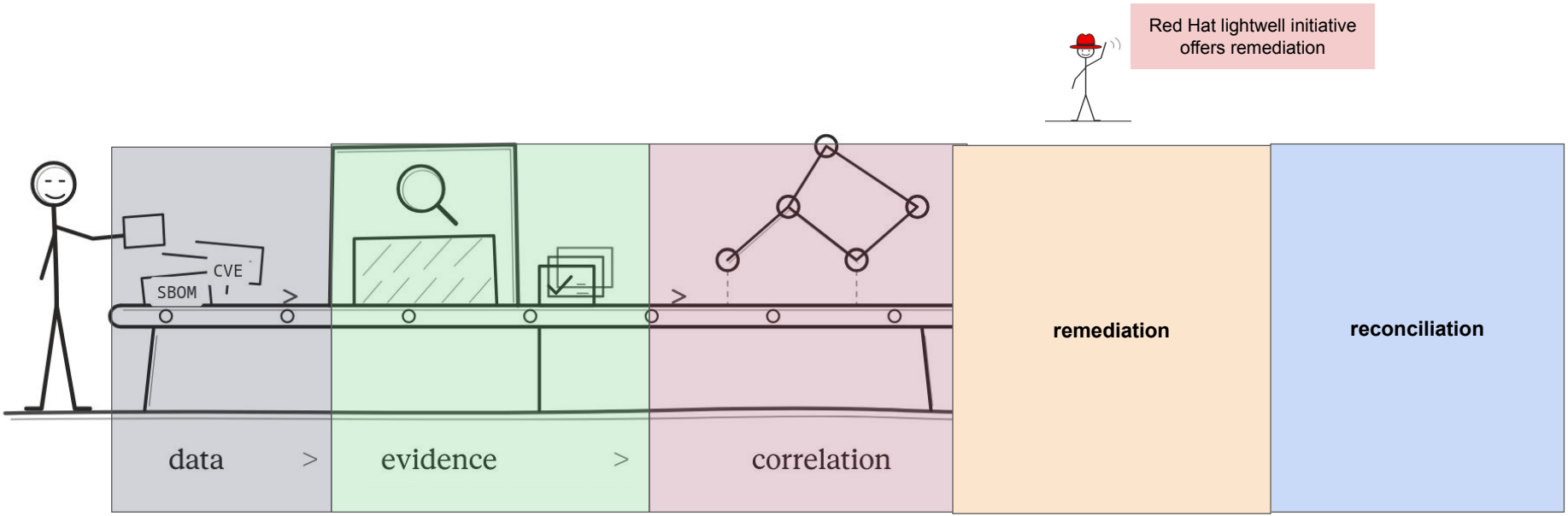
Remediation Category

-- Select (optional) --

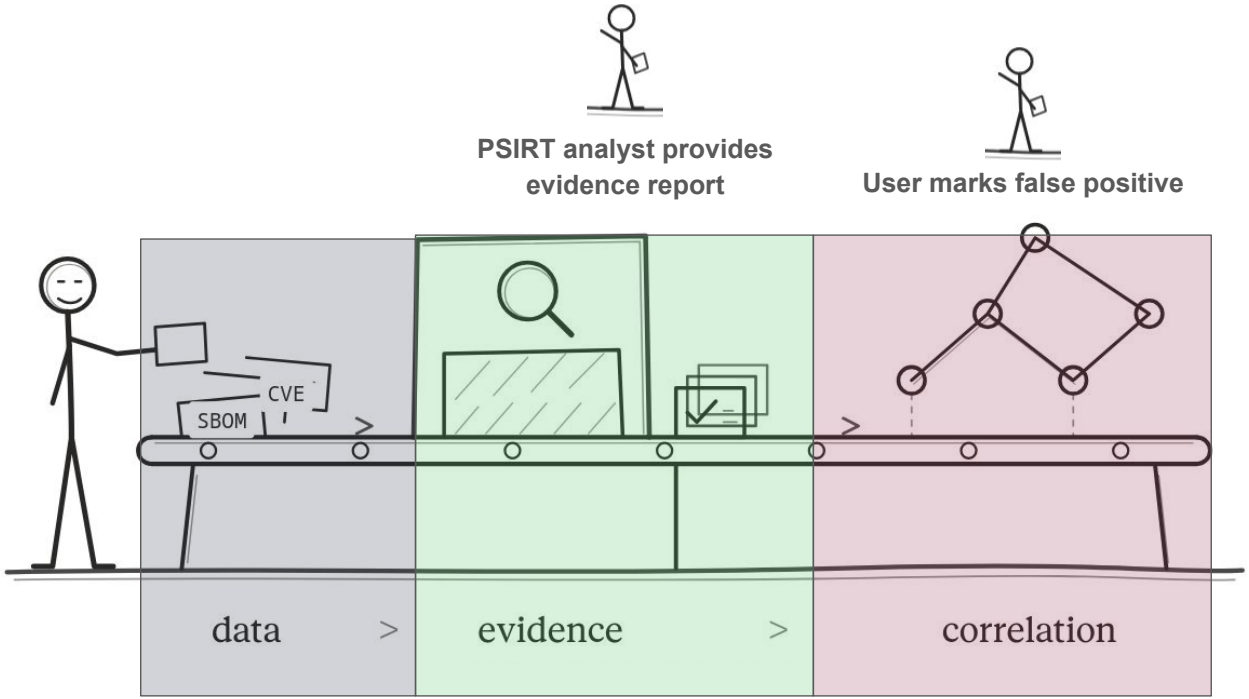
Impact Statement

Optional impact or threat description...

Users want remediation



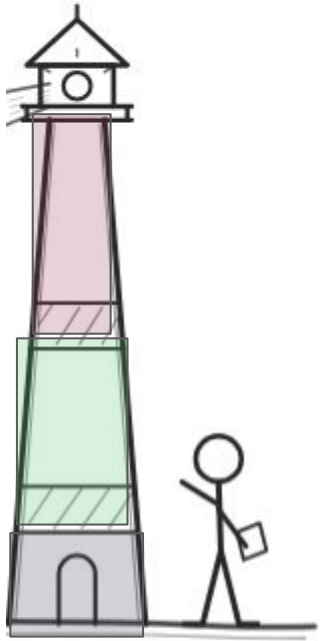
Deterministic ‘lighthouse’in the AI Storm



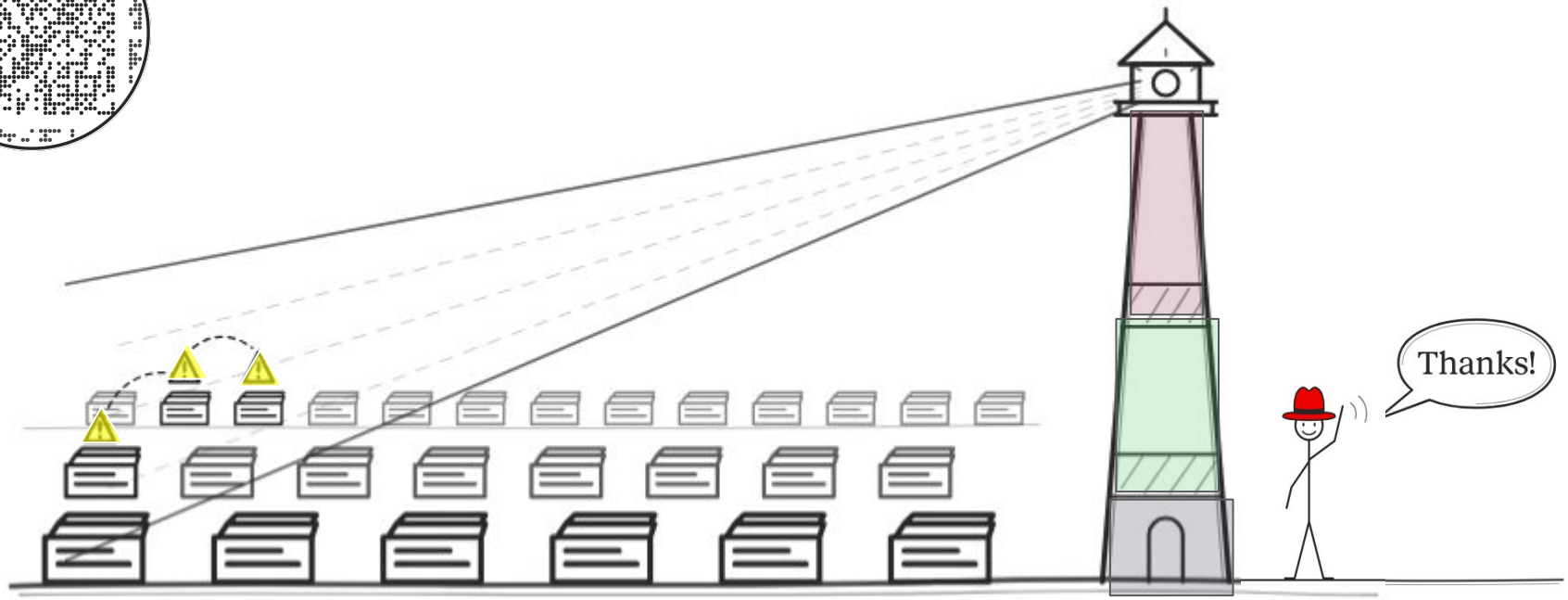


Red Hat EI reachability analysis

False positive analysis



Questions



https://github.com/guacsec/trustify